

# Graphs

in the "real world"



- Networks are graphs
  - Information: WWW, citation, ...
  - Social: co-actor, dating, messenger, communities, ...
  - Technological: Internet, power grids, airline routes, ...
  - Biological: Neural networks, food web, blood vessels, ...
- Object hierarchies are graphs
- Circuit layouts are graphs
- Computer programs are graphs

# Undirected Graph $G=(V,E)$ (or $G=(N,A)$ )

$V$ : Set of vertices (nodes...)

Edge  $\{a,b\}$  is an unordered pair of nodes

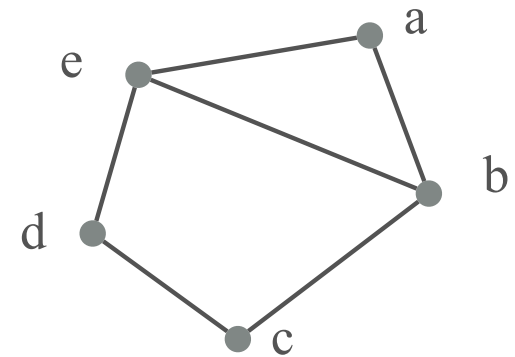
$E \subset V \times V$ : Set of vertices (arcs...)

Undirected graph:  $G=(V,E)$

- Typically, don't allow  $\{a,a\}$  in  $E$

**Vertex Set:  $V = (a,b,c,d,e)$**

**Edge Set:  $E = ( \{a,b\}, \{b,c\}, \{c,d\}, \{d,e\}, \{e,a\}, \{e,b\} )$**



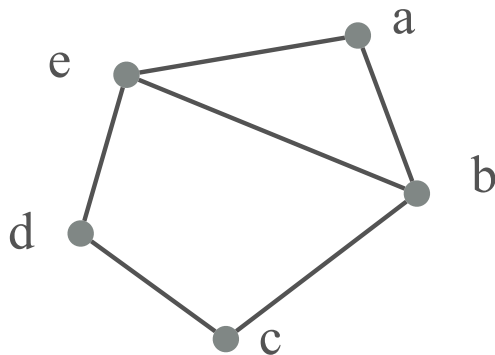
degree = 3

# Undirected Graph $G(V,E)$

Maximum number of edges:  $|E| \leq \binom{|V|}{2} = \frac{|V|(|V|-1)}{2} = O(|V|^2)$

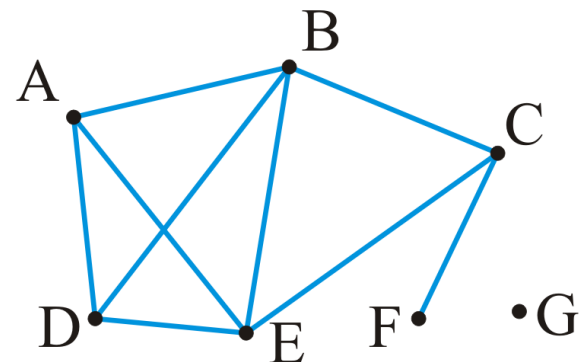
**Degree** of vertex: number of adjacent vertices

Vertex  $a$  **adjacent** to vertex  $b \iff \{a,b\} \in E$



degree(b) = 3

degree(c) = 2



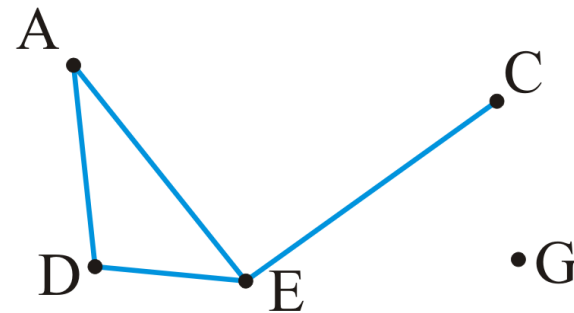
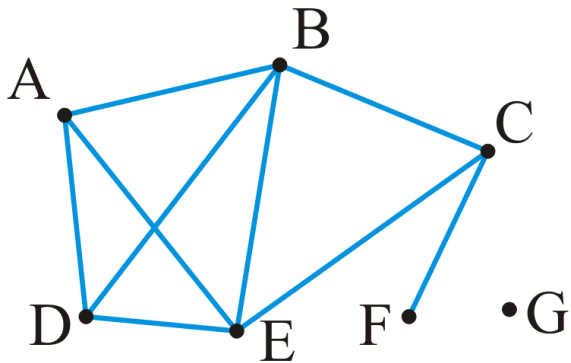
degree(E) = 4

degree(G) = 0

## Subgraphs $(V', E')$ of $(V, E)$

$(V', E')$  is a subgraph of  $(V, E)$  if and only if

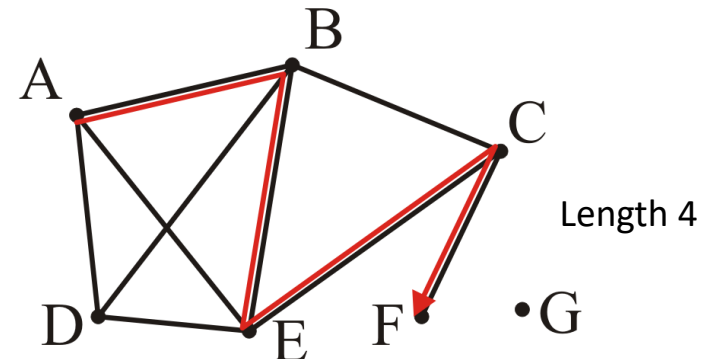
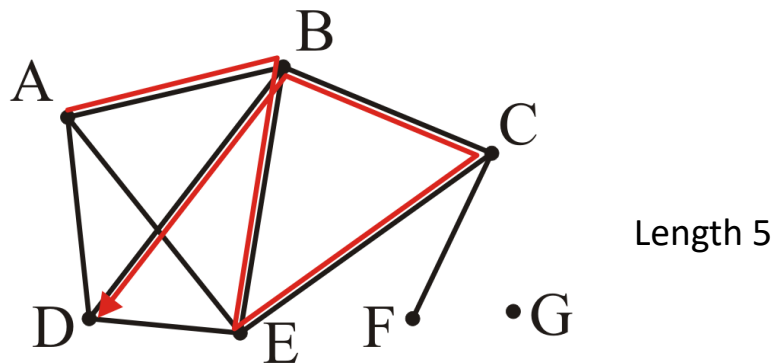
- $V' \subset V$ , and  $E' \subset E$
- If  $\{a, b\} \in E' \Rightarrow a, b \in V'$



## Paths in(V,E)

A path in an undirected graph is an ordered sequence of vertices  $(v_0, v_1, v_2, \dots, v_k)$  such that:

- $\{v_{j-1}, v_j\}$  is an edge in  $E$  for  $j = 1, \dots, k$
- Termed a path from  $v_0$  to  $v_k$
- The length of this path is  $k$ : number of edges

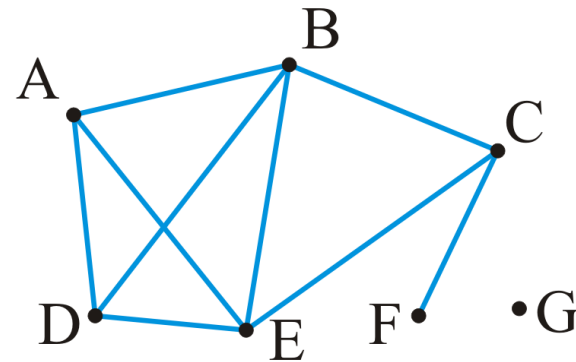


## Simple paths in $(V,E)$

Simple path: no repeated vertices other than perhaps, the first and last vertices

Simple cycle: simple path with first and last vertices the same

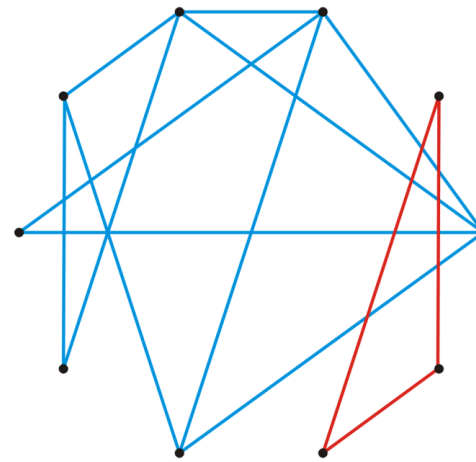
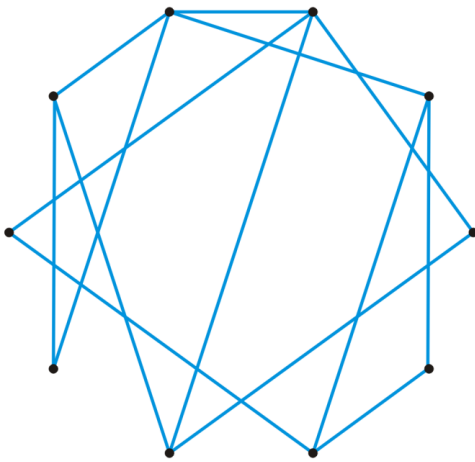
- Example:  $(A,B,C,E)$  simple path
- $(A,B,E,C,B,D)$  Not simple path
- $(A,B,D,A)$  simple cycle



# Connectedness

Vertices  $a, b$  in  $V$  are connected if there exists a path from  $a$  to  $b$

Graph  $(V, E)$  is connected if, for every  $a, b$  in  $V$ ,  $a$  and  $b$  are connected



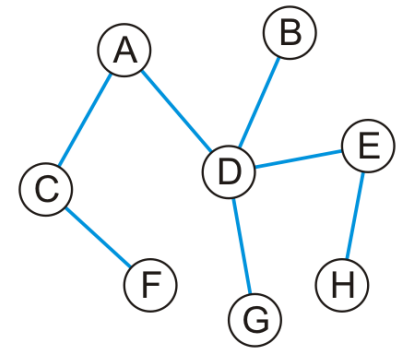
# Trees

Graph(V,E) is a tree if it is connected, and

- there is only one path between any two vertices a, b
- Equivalently, there are no cycles in the graph

Trees:  $|E| = |V| - 1$

- Adding one more edge to a tree creates a cycle
- Removing an edge from a tree disconnects the graph
- Can convert to rooted tree picking any vertex as root





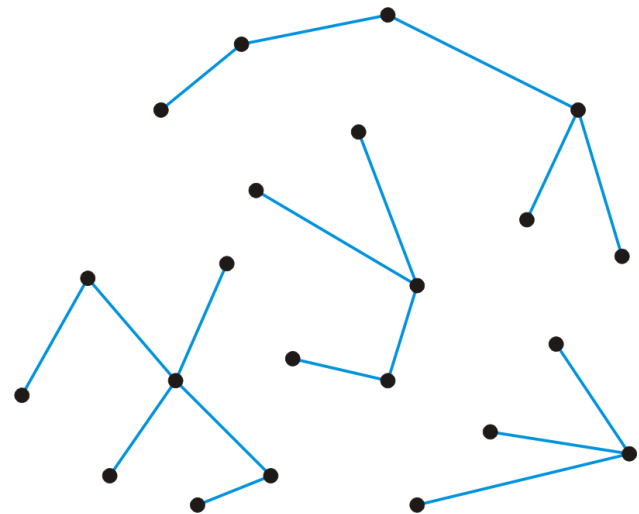
# Forests

Forest is a graph  $(V,E)$  with no cycles

- Forest can be multiple disjoint trees

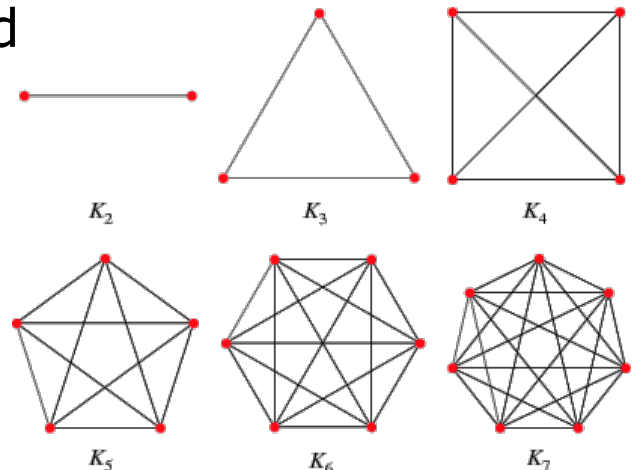
Properties:

- $|E| < |V|$
- Number of trees =  $|V| - |E|$
- Removing edge: adds one more tree



# Some Specific Graphs

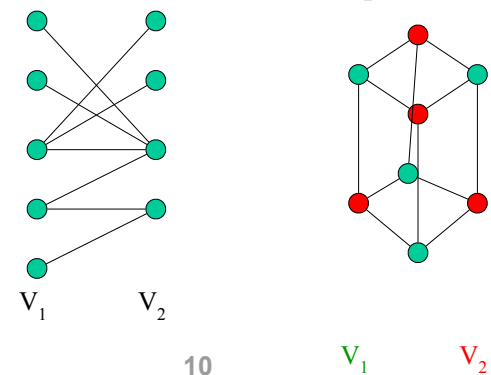
Complete graph  $K_n$ : undirected graph with  $n$  vertices, and an edge between every pair of vertices



Bipartite graph: undirected graph where  $V$  can be partitioned into disjoint, nonempty sets  $V_1$  and  $V_2$  such that

every edge connects a vertex in  $V_1$  to a vertex in  $V_2$

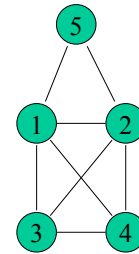
Equivalent: every cycle has even length!



## Results in Undirected Graphs

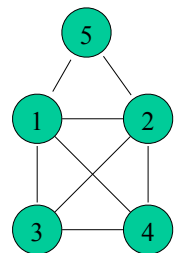
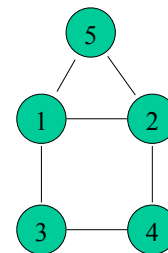
Let  $(V,E)$  be undirected. Then,  $\sum_{k \in V} \text{degree}(k) = 2|E|$

- Every edge connects 2 vertices...  
—> must have even number of odd-degree vertices



A path is **Eulerian** if it covers every edge in the graph exactly once

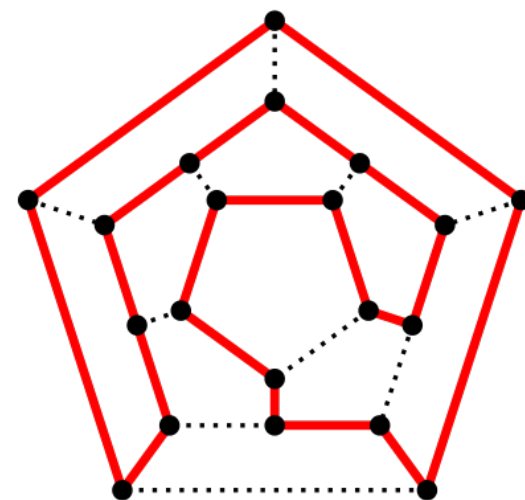
- Draw it without lifting pencil
- **Theorem**: a connected graph has an Eulerian cycle if and only if every node has even degree



## Results in Undirected Graphs

A path is **Hamiltonian** if it covers every vertex in the graph exactly once; it is a cycle if it covers every vertex but the start and end vertices exactly once

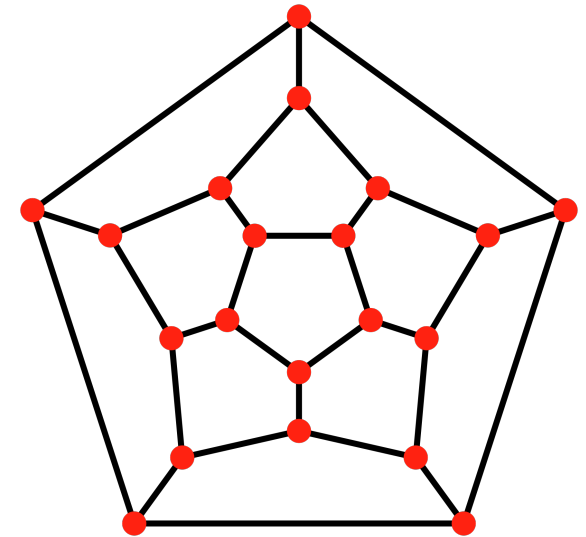
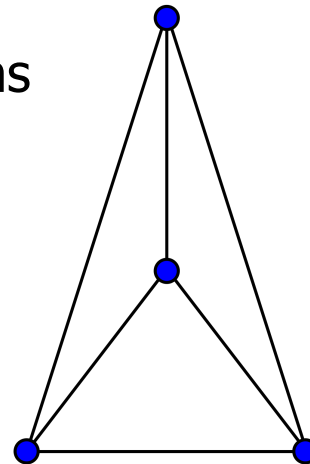
- **Theorem:** A graph with  $n$  vertices has Hamiltonian cycle if degree of every vertex is at least  $n/2$
- **Theorem:** For a graph  $G$  with  $n \geq 3$  vertices, and  $\text{degree}(a) + \text{degree}(b) \geq n$  whenever  $a, b$  are not adjacent, then  $G$  has a Hamiltonian cycle
  - Finding a Hamiltonian cycle is a hard problem!



# Planar Graphs

A graph  $G$  is planar if it can be drawn in the plane so that no edges cross

- Useful for VLSI, circuit design
- **Euler's Formula:** a planar graph divides the plane into regions called faces. The number of regions  $r = m - n + 2$
- Sufficient conditions for planar graphs
  - $m \leq 3n - 6$
  - $r \leq 2n - 4$



# Directed Graph $G=(V,E)$

$V$  is the set of vertices

An edge  $e = (a,b)$  is an ordered pair from  $a$  to  $b$ , where  $a, b$  in  $V$

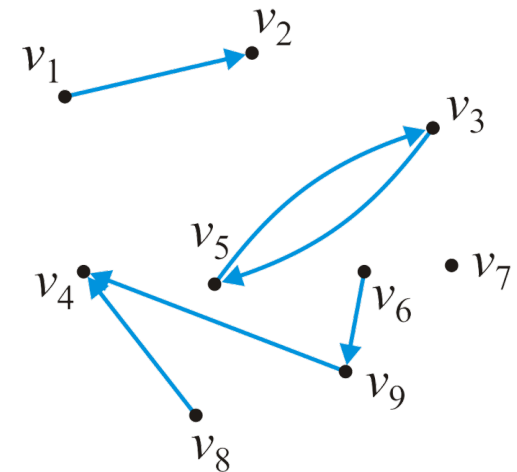
Note: edge  $(b, a)$  is different from edge  $(a, b)$

Example: streets, ...

Max. no. edges =  $|V|^2$  (allow self loops)

**Out-degree** of a vertex: no. edges leaving vertex

**In-degree** of a vertex: no. edges entering vertex



$$E = \{(v_1, v_2), (v_3, v_5), (v_5, v_3), (v_6, v_9), (v_8, v_4), (v_9, v_4)\}$$

## Sources, Sinks, Paths

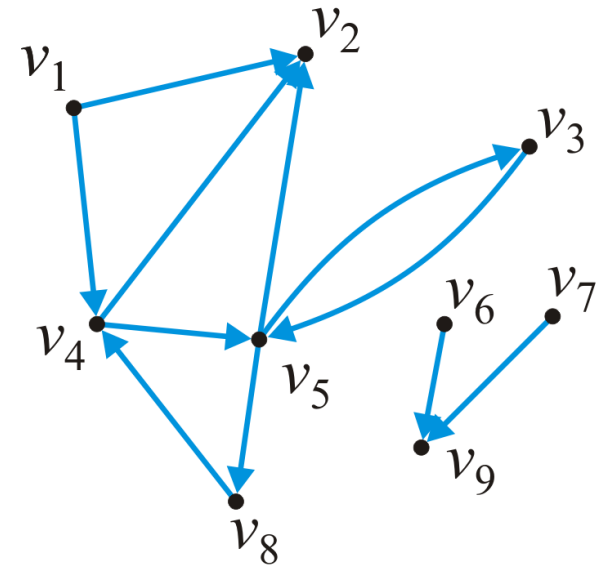
Vertex with in-degree 0: **Source** ( $V_1, V_6, V_7$ )

Vertex with out\_degree 0: **Sink** ( $V_9$ )

**Path** in directed graph  $(V, E)$  is an ordered sequence of vertices  $(v_0, v_1, v_2, \dots, v_k)$  such that  $(v_{j-1}, v_j) \in E$  for  $j = 1, \dots, k$

**Simple path:** no repeated nodes in path

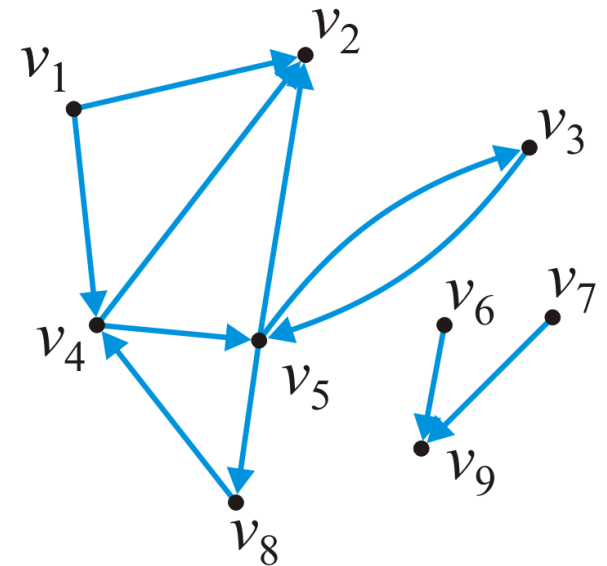
**Cycle:** path with  $v_0 = v_k$ . **Simple cycle:** no repeated nodes except start-end



# Connectedness in Directed Graphs

Vertex  $a$  is connected to vertex  $b$  if there exists a path from  $a$  to  $b$  (directed path)

- Graph  $(V,E)$  is **strongly connected** if there exists directed path between any two vertices
- Graph  $(V,E)$  is **weakly connected** if there exists undirected path between any two vertices
  - The sub-graph  $\{v_3, v_4, v_5, v_8\}$  is strongly connected
  - The sub-graph  $\{v_1, v_2, v_3, v_4, v_5, v_8\}$  is weakly connected

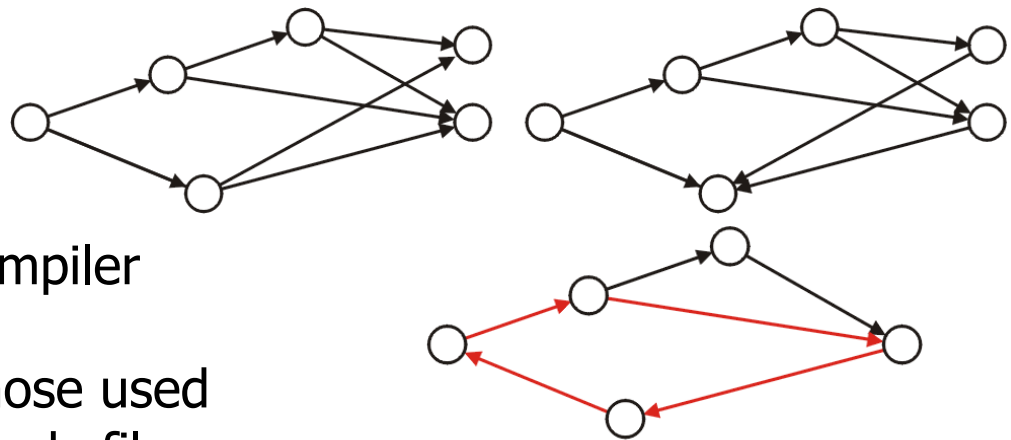




# Directed Acyclic Graphs

Directed acyclic graph (DAG): Graph  $(V,E)$  with no directed cycles

- Represents partial order
- DAGs appear often
  - Parse tree constructed by a compiler
  - Dependency graphs such as those used in instruction scheduling and makefiles
  - Dependency formed by inheritance relationships in object-oriented programming languages



# Representations of graphs

- Adjacency matrix:

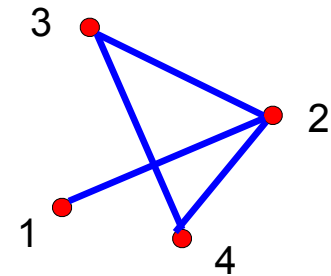
- $A = [a_{ij}]$ , where

$$a_{ij} = \begin{cases} 1 & \text{if } (i, j) \in E \\ 0 & \text{if } (i, j) \notin E \end{cases}$$

- Symmetric for undirected graphs

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix} \begin{matrix} \cdots 1 \\ \cdots 2 \\ \cdots 3 \\ \cdots 4 \\ \vdots \\ \vdots \end{matrix}$$

1   2   3   4



# Adjacency List

- Each vertex has list of neighbor nodes

- For directed graphs, out-list

- Used in MATLAB for sparse matrices

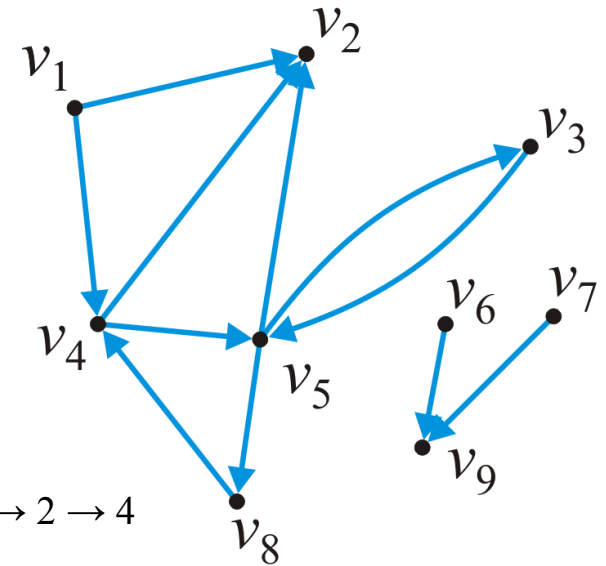
- Can be implemented as arrays

- Forward-Star

- or linked lists

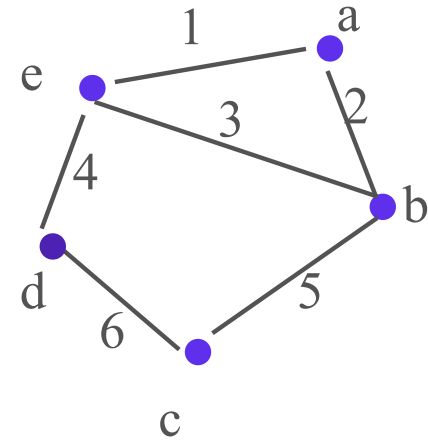
- $|V| + |E|$  for directed graphs

- Must store 2 edges for undirected



```
1 • → 2 → 4
2 •
3 • → 5
4 • → 2 → 5
5 • → 2 → 3 → 8
6 • → 9
7 • → 9
8 • → 4
9 •
```

V vertices  
E edges

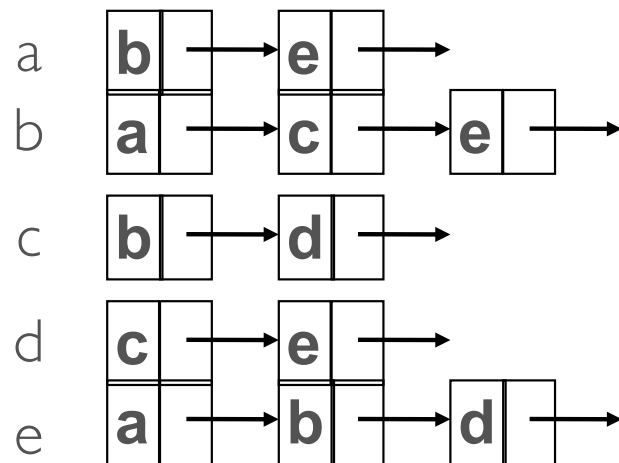


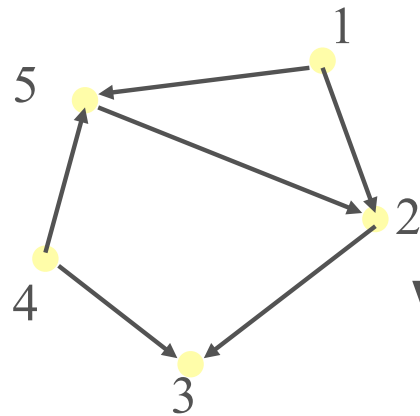
1. Edge list: {a,b}, {b,c}, {c,d}, {d,e}, {e,a}, {e,b}

2. Adjacency list

| Vertex | Adjacencies |
|--------|-------------|
| a      | b, e        |
| b      | a, c, e     |
| c      | b, d        |
| d      | c, e        |
| e      | a, b, d     |

Linked List Form





## Forward Star Representation

Vertex Array First[ I ]

i = 1

1

i = 2

3

i = 3

4

i = 4

4

i = 5

6

i = 6

7

Edges[ j ]

j = 1

2

j = 2

5

j = 3

3

j = 4

3

j = 5

5

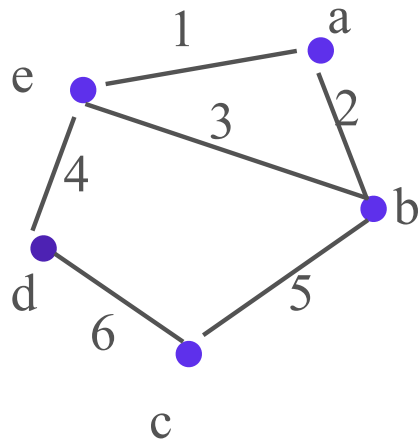
j = 6

2

j = 7

-1

Just the adjacency list put end to end in the arc array!



3. Adjacency matrix:

A=

|   | a | b | c | d | e |
|---|---|---|---|---|---|
| a | 0 | 1 | 0 | 0 | 1 |
| b | 1 | 0 | 1 | 0 | 1 |
| c | 0 | 1 | 0 | 1 | 0 |
| d | 0 | 0 | 1 | 0 | 1 |
| e | 1 | 1 | 0 | 1 | 0 |

4. Incidence matrix:

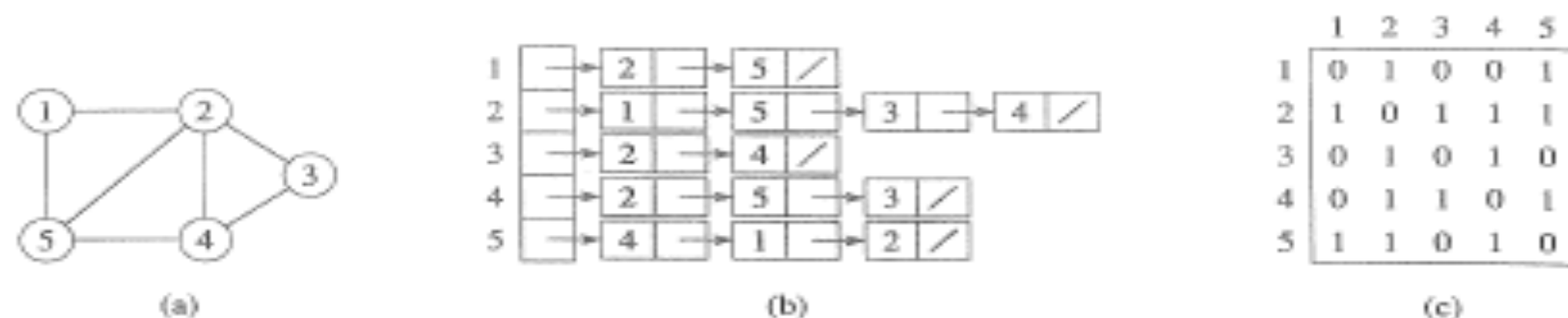
Matrix of vertex rows, edges columns

Directed: -1 in start of edge, 1 in end

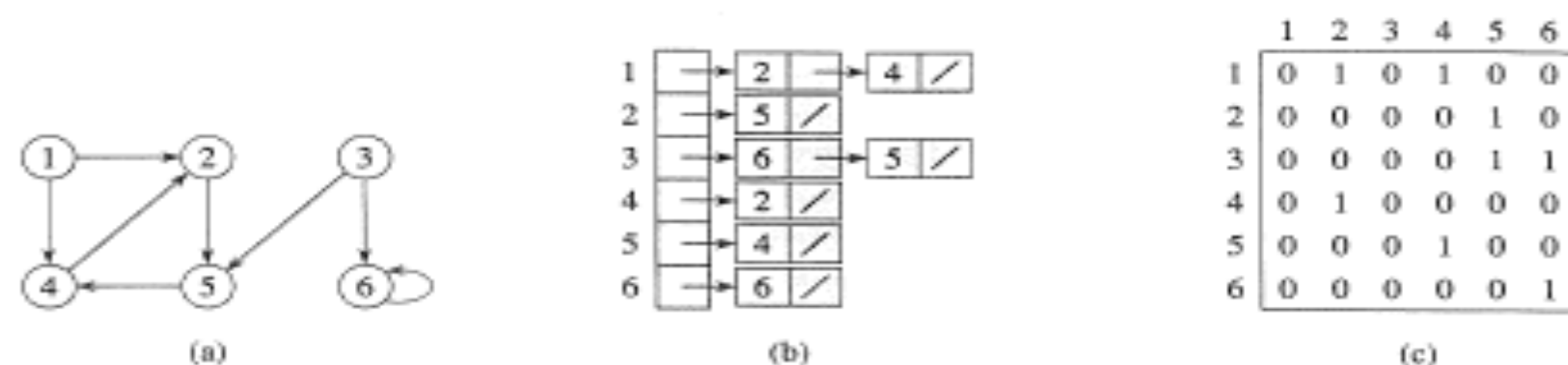
Undirected: 1 in both

M=

|   | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| a | 1 | 1 | 0 | 0 | 0 | 0 |
| b | 0 | 1 | 1 | 0 | 1 | 0 |
| c | 0 | 0 | 0 | 0 | 1 | 1 |
| d | 0 | 0 | 0 | 1 | 0 | 1 |
| e | 1 | 0 | 1 | 1 | 0 | 0 |



**Figure 22.1** Two representations of an undirected graph. (a) An undirected graph  $G$  having five vertices and seven edges. (b) An adjacency-list representation of  $G$ . (c) The adjacency-matrix representation of  $G$ .



**Figure 22.2** Two representations of a directed graph. (a) A directed graph  $G$  having six vertices and eight edges. (b) An adjacency-list representation of  $G$ . (c) The adjacency-matrix representation of  $G$ .

## Graph Traversals

- Traversals of graphs are also called *searches*
- We can use either breadth-first or depth-first traversals
  - Breadth-first requires a queue
  - Depth-first requires a stack
- In each case, we will have to track which vertices have been visited requiring  $\Theta(|V|)$  memory
- The time complexity cannot be better than and should not be worse than  $\Theta(|V| + |E|)$ 
  - Connected graphs simplify this to  $\Theta(|E|)$
  - Worst case:  $\Theta(|V|^2)$



## Breadth-First Search: Uses a Queue

1. Mark all vertices as unvisited, parents as NULL, depth as -1
  2. Choose any unvisited vertex, mark it as **visited** and enqueue it onto queue
  3. While the queue is not empty:
    - Dequeue top vertex  $v$  from the queue. Do work to be done on that vertex
      - If  $\text{parent}[v] == \text{NULL}$ , set depth to 0; otherwise, set depth to  $\text{depth}[\text{parent}[v]] + 1$
      - For each vertex adjacent to  $v$  (e.g. in out list) that has not been visited: Mark it visited, mark its parent as  $v$ , and enqueue it
      - Mark  $v$  as done
  4. If there are unvisited vertices, choose any unvisited vertex, mark it as **visited**, enqueue it and repeat step 3
- This can handle graphs that are not connected
    - Marking as visited avoids cycles
    - Complexity:  $O(\#V + \#E)$ , reduces to  $O(\#E)$  if strongly connected
    - Size of queue is  $O(\#V)$

## Depth-First Search

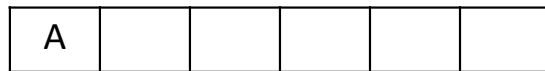
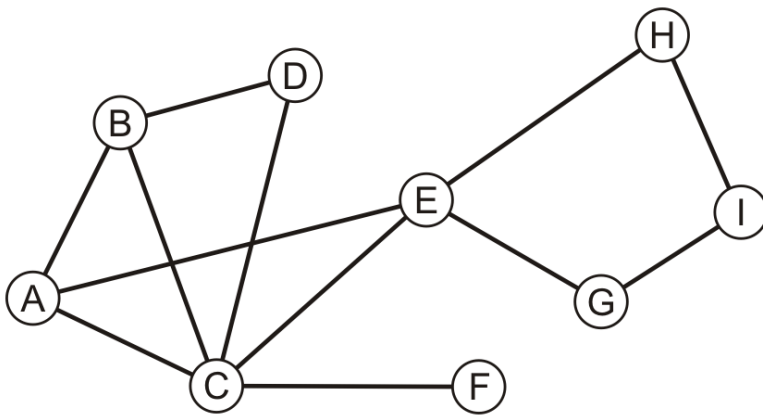
Recursive implementation:

1. Mark all vertices as unvisited; mark all parents as NULL
2. While there are vertices marked as unvisited:
  - Select unvisited vertex  $v$ , mark as visited:
  - Do DFS(vertex)

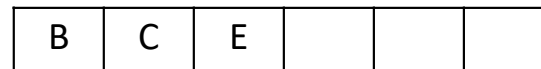
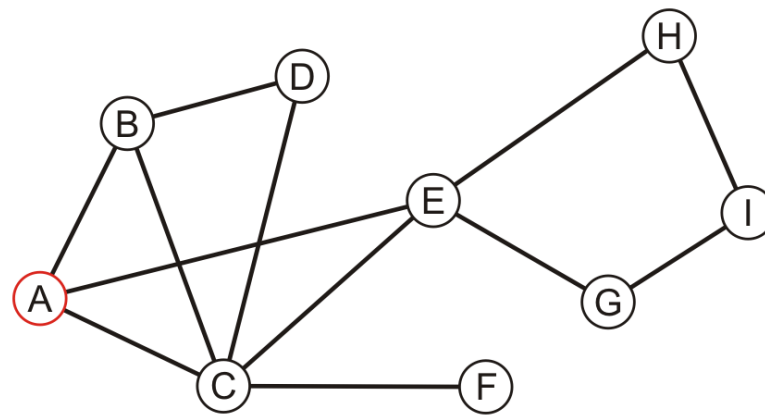
DFS(vertex):

- For neighbors of vertex
  - If neighbor is unvisited, mark as visited and do DFS(neighbor)
- This can handle graphs that are not strongly connected
  - Marking as visited avoids cycles
  - Complexity:  $O(\#V + \#E)$ , reduces to  $O(\#E)$  if strongly connected
  - Size of queue is  $O(\#V)$

## Example: BFS



Queue

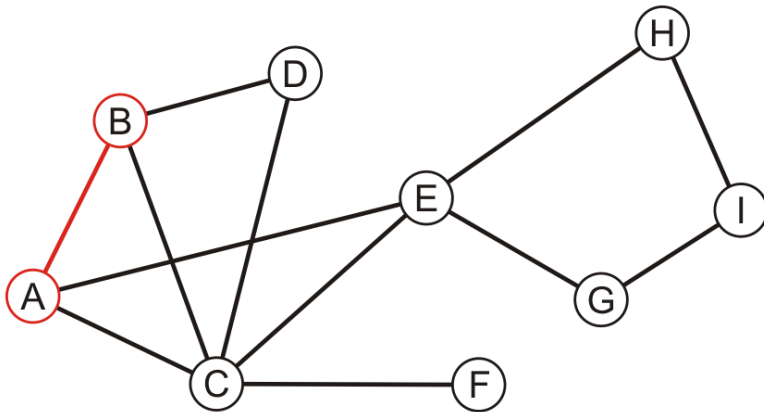


## Example

Performing a breadth-first traversal:

- Pop B and push D

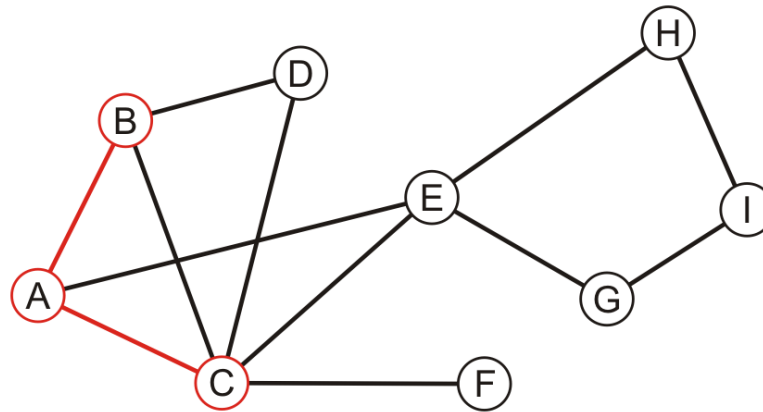
Done: A, B



|   |   |   |  |  |  |
|---|---|---|--|--|--|
| C | E | D |  |  |  |
|---|---|---|--|--|--|

Pop C and push F

A, B, C

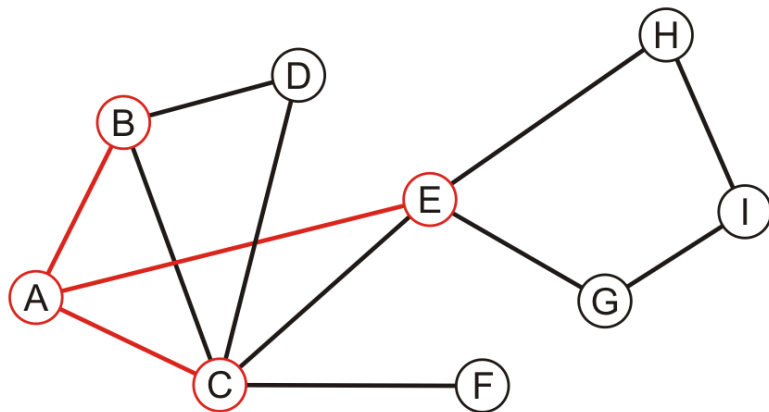


|   |   |   |  |  |  |
|---|---|---|--|--|--|
| E | D | F |  |  |  |
|---|---|---|--|--|--|

## Example

Performing a breadth-first traversal:

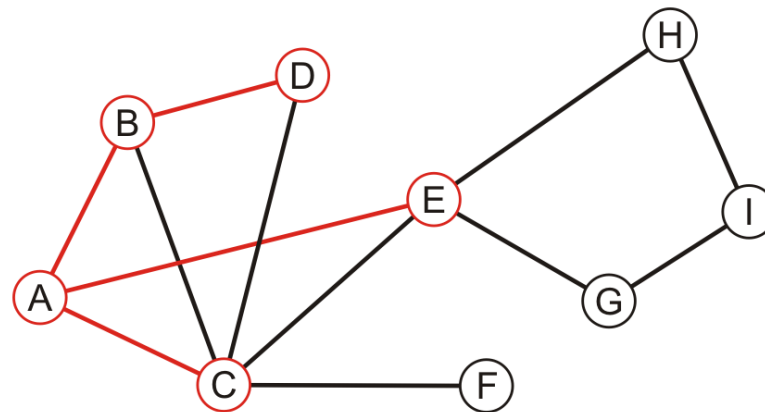
- Pop E and push G,
- Done: A, B, C, E



|   |   |   |   |  |  |
|---|---|---|---|--|--|
| D | F | G | H |  |  |
|---|---|---|---|--|--|

Pop D

A, B, C, E, D

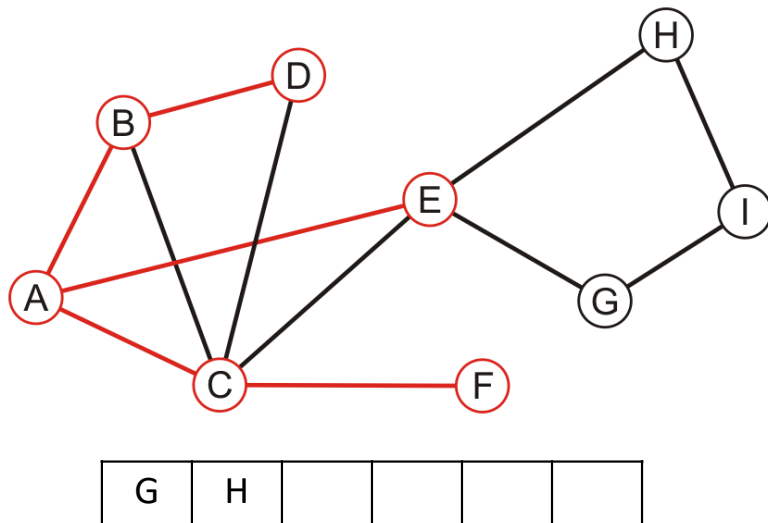


|   |   |   |  |  |  |
|---|---|---|--|--|--|
| F | G | H |  |  |  |
|---|---|---|--|--|--|

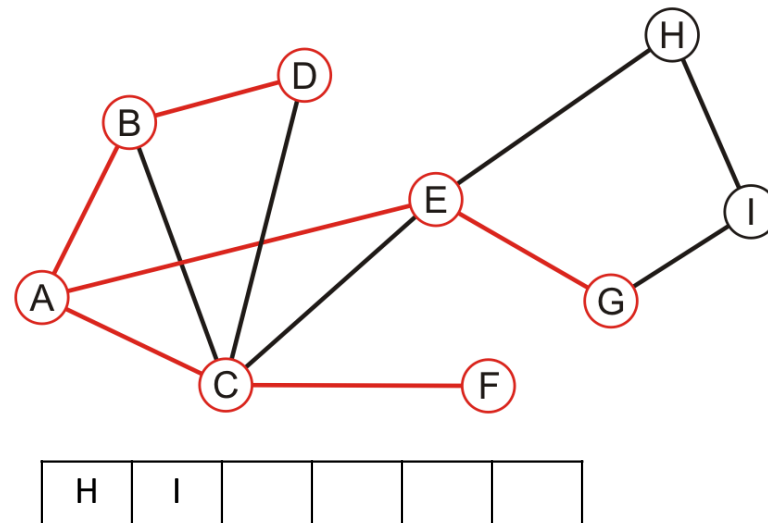
## Example

Performing a breadth-first traversal:

- Pop F
- Done: A, B, C, E, D, F



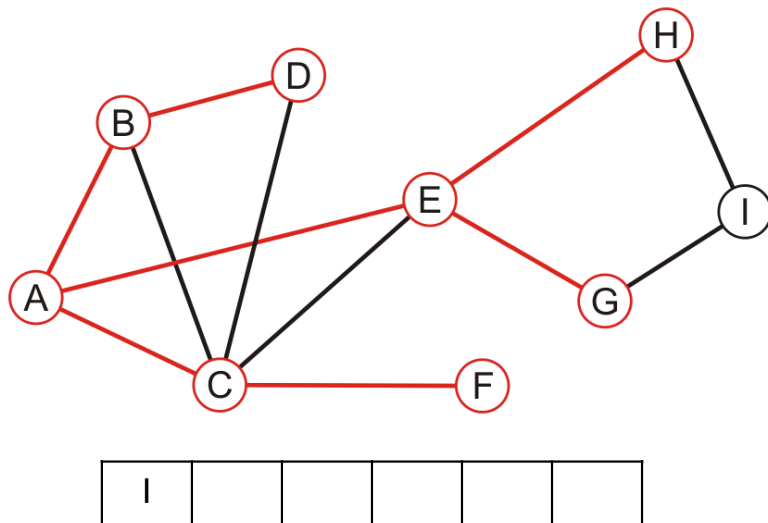
Pop G, push I  
A, B, C, E, D, F, G



## Example

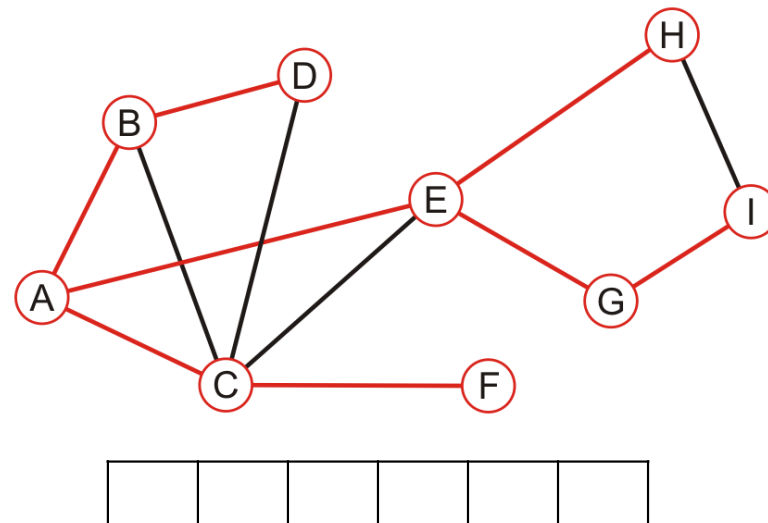
Performing a breadth-first traversal:

- Pop H
- Done: A, B, C, E, D, F, G, H



Pop I

A, B, C, E, D, F, G, H, I



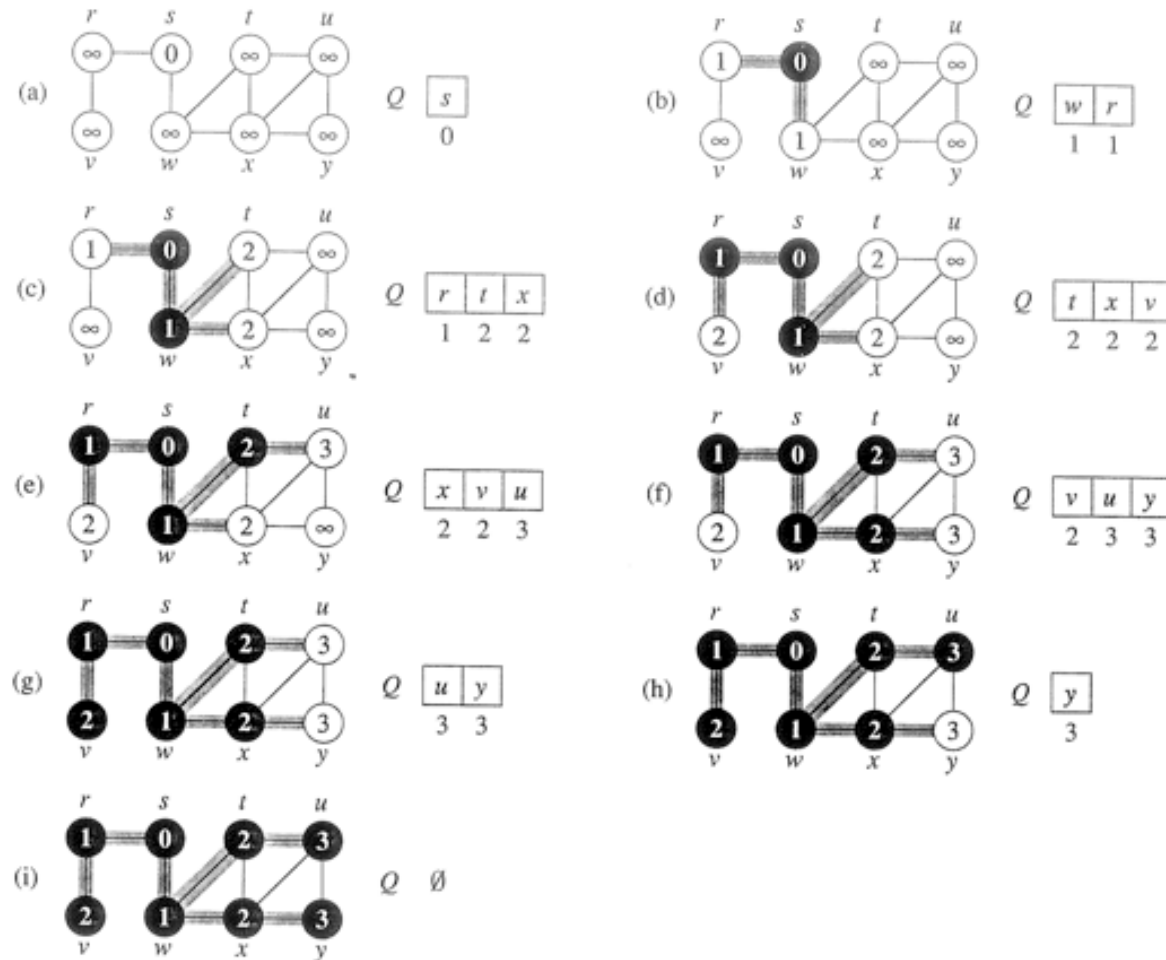


Figure 22.3 The operation of BFS on an undirected graph. Tree edges are shown shaded as they are produced by BFS. Within each vertex  $u$  is shown  $d[u]$ . The queue  $Q$  is shown at the beginning of each iteration of the while loop of lines 10–18. Vertex distances are shown next to vertices in the queue.

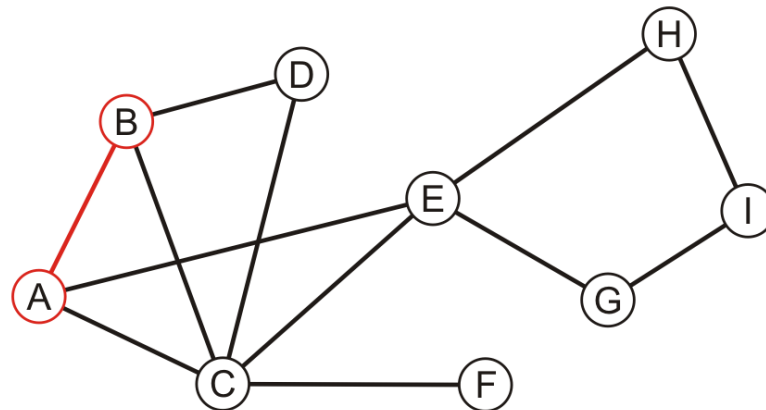
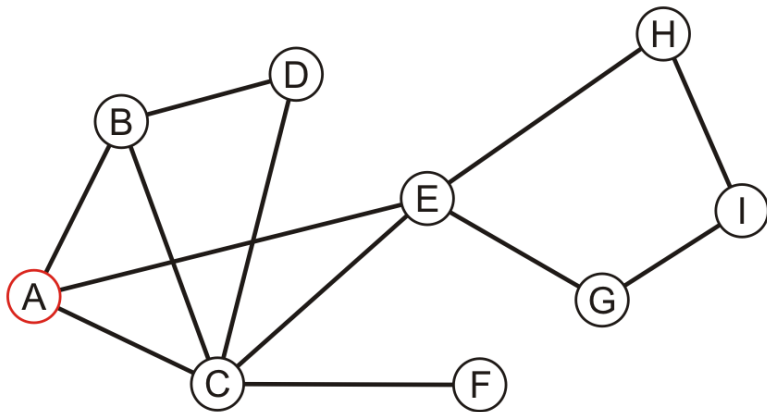


## Example

Performing a recursive depth-first traversal:

- Insert A: Visited: A, B  
Stack: A, B

Examine B: Visited A, B, C,  
Stack A, B, C

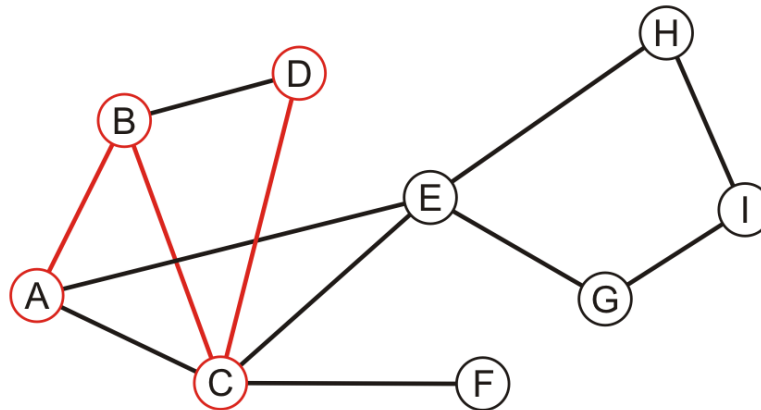
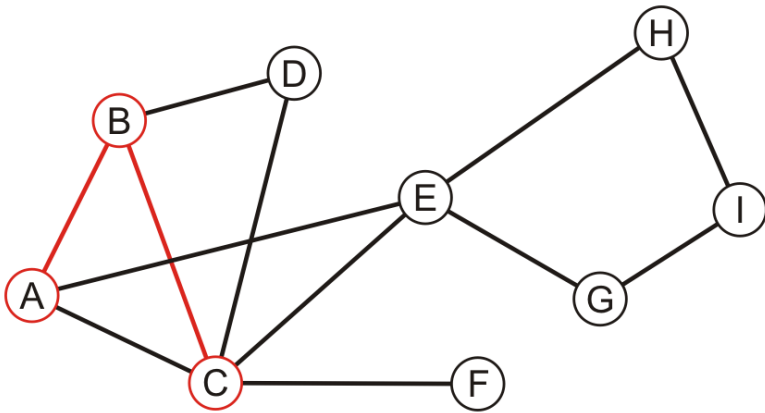


## Example

Performing a recursive depth-first traversal:

- Examine C: Visited: A, B, C, D  
Stack: A, B, C, D

Pop D: Visited A, B, C, D  
Stack A, B, C

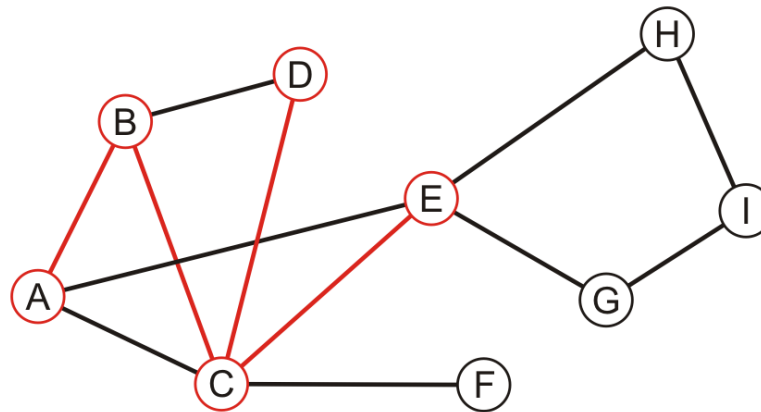
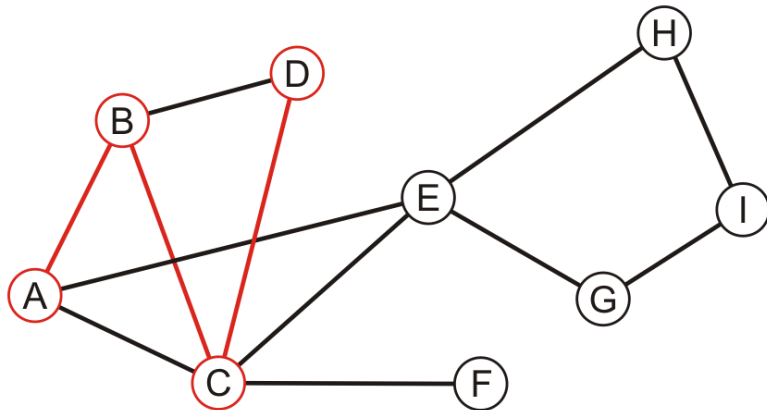


## Example

Performing a recursive depth-first traversal:

- Examine C: Visited: A,B,C,D,E  
Stack: A,B,C,E

Examine E: Visited A,B,C,D,E,G  
Stack A,B,C,E,G

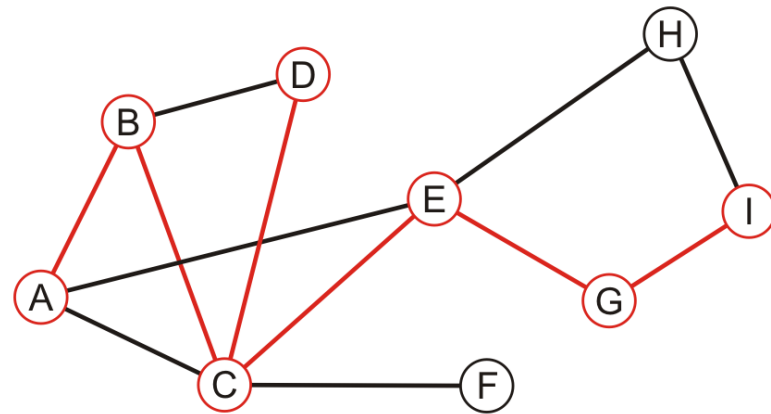
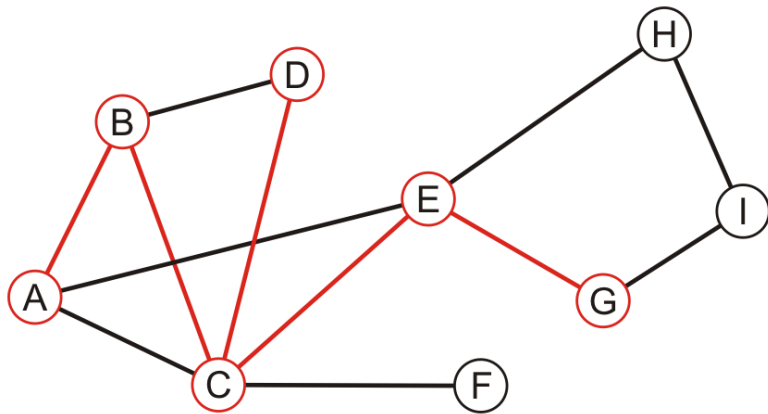


## Example

Performing a recursive depth-first traversal:

- Examine G: Visited: A,B,C,D,E,G,I  
Stack: A,B,C,E,G,I

Examine I: Visited A,B,C,D,E,G,I,H  
Stack A,B,C,E,G,I,H



## Example

Performing a recursive depth-first traversal:

– Pop H: Visited: A,B,C,D,E,G,I,H

Stack: A,B,C,E,G,I

Pop G, then E

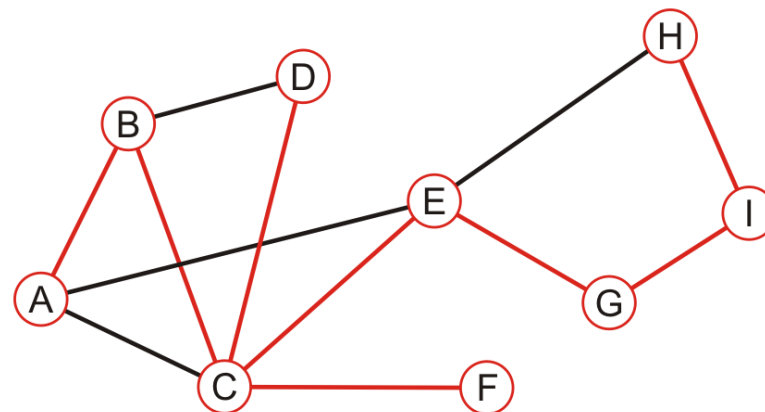
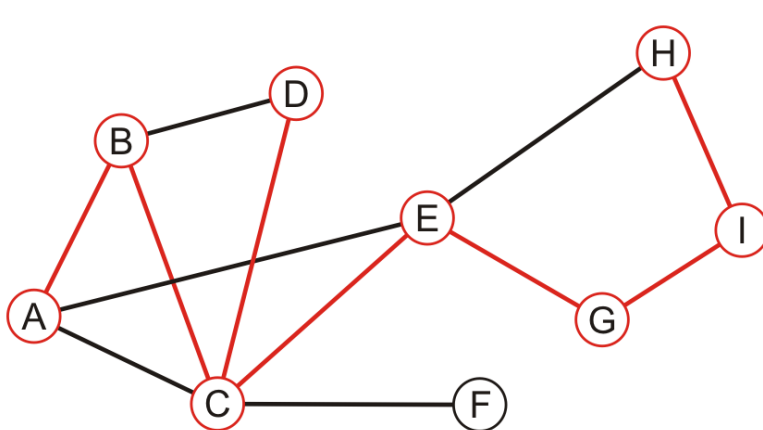
Inspect C: Visited A,B,C,D,E,G,I,H,F

Stack: A,B,C,F

Pop I: Visited A,B,C,D,E,G,I,H

Stack A,B,C,E,G,

Pop F, C, B, A. Done.

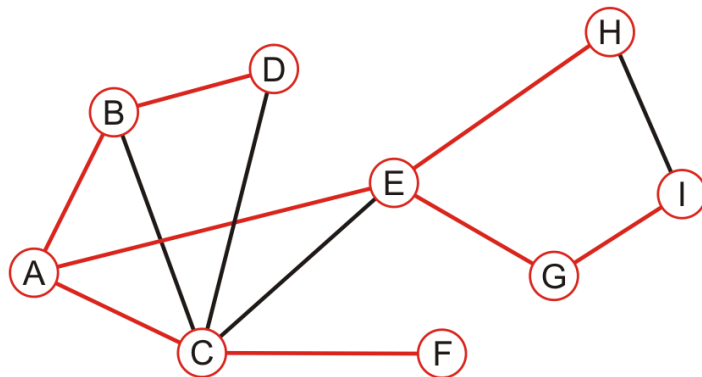


## Comparison

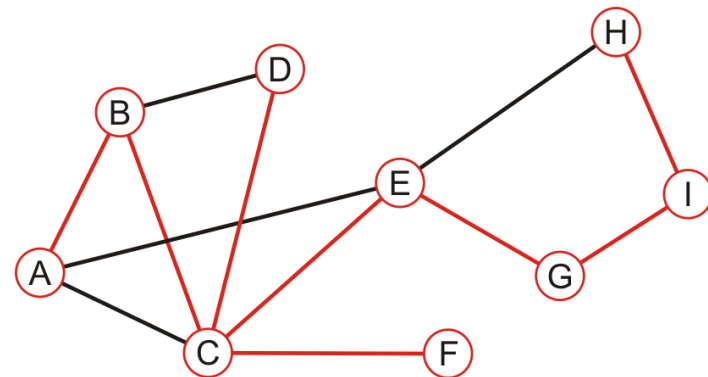
The order in which vertices can differ greatly

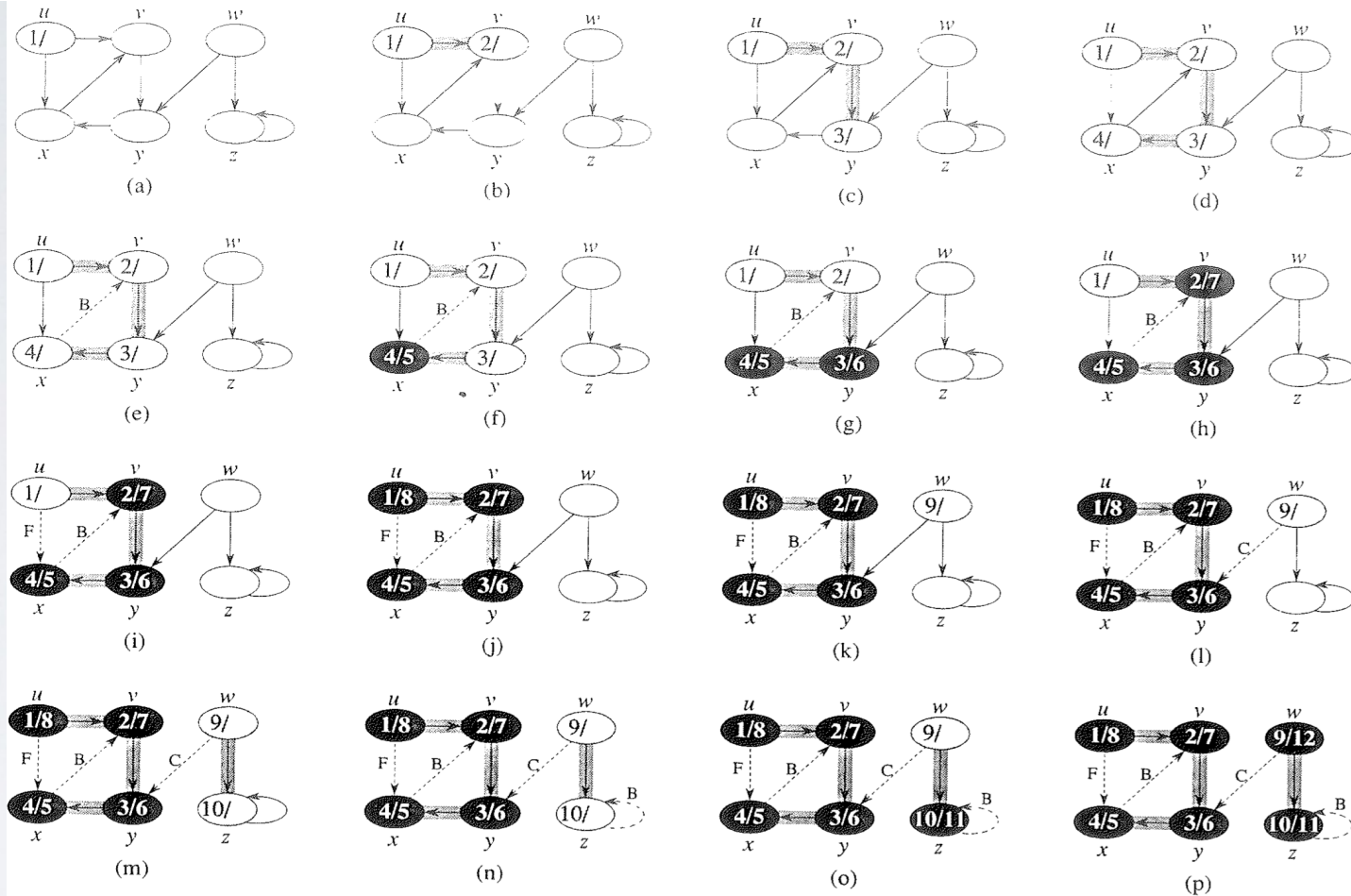
- An iterative depth-first traversal may also be different again

A, B, C, E, D, F, G, H, I



A, B, C, D, E, G, I, H, F





**Figure 22.4** The progress of the depth-first-search algorithm DFS on a directed graph. As edges are explored by the algorithm, they are shown as either shaded (if they are tree edges) or dashed (otherwise). Nontree edges are labeled B, C, or F according to whether they are back, cross, or forward edges. Vertices are timestamped by discovery time/finishing time.

## Applications

Applications of graph traversals include:

- Determining connectedness and finding connected sub-graphs
- Determining the path length from one vertex to all others
- Testing if a graph is bipartite
- Branch and bound search
- Topological Sort
- ...

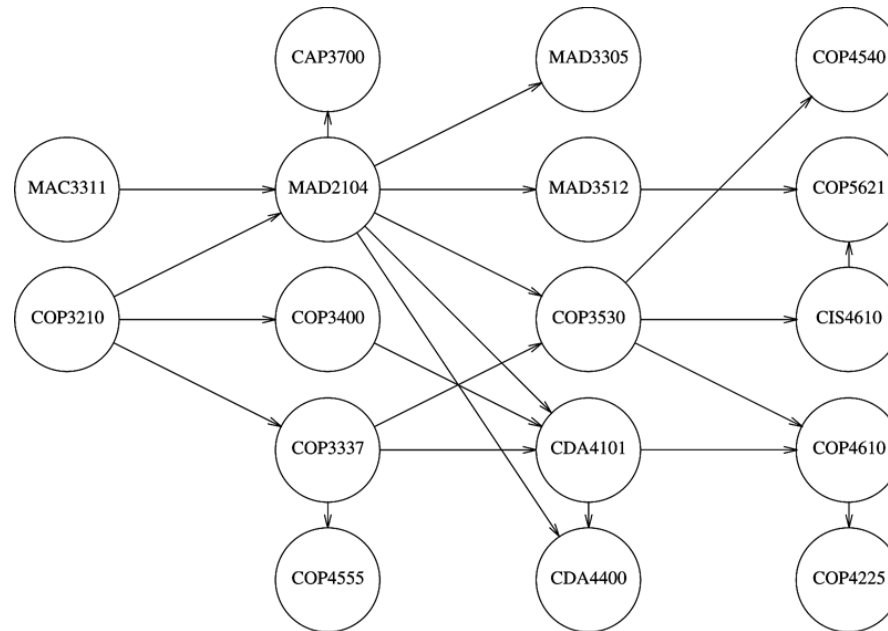


# Topological Sort

- Application
  - Given a number of tasks, there are often a number of constraints between the tasks:
    - task A must be completed before task B can start
  - These tasks together with the constraints form a **directed acyclic graph**
  - A topological sort of the graph gives an order in which the tasks can be scheduled while still satisfying the constraints

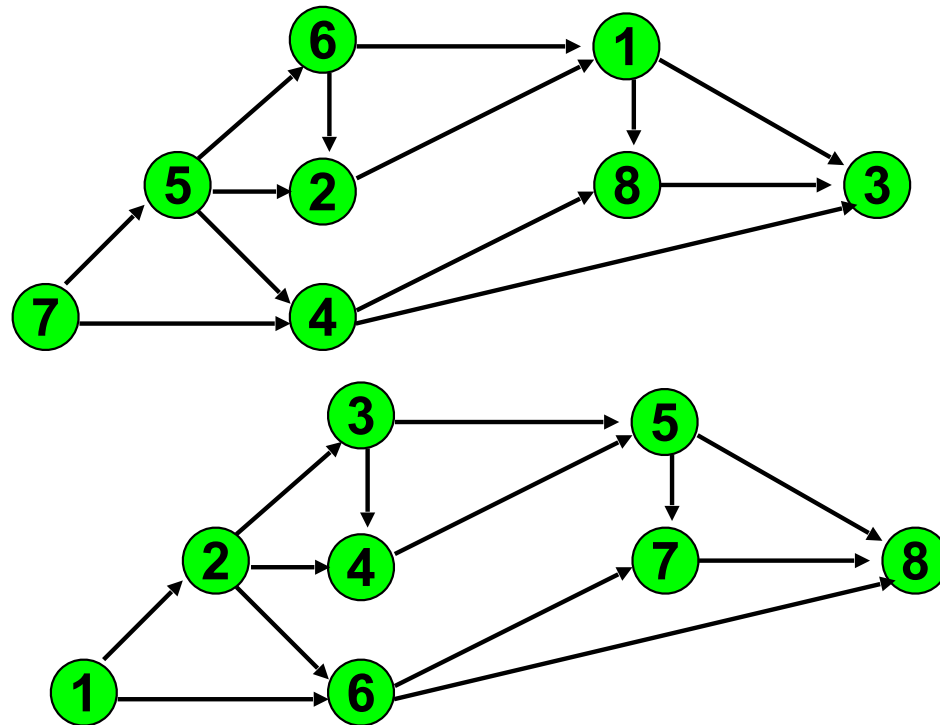
# Topological Sort

- Course prerequisite structure represented in an acyclic graph



# Topological Sort

- Topological Sort (Relabel): Given a directed acyclic graph (DAG), relabel the vertices such that every directed edge points from a lower-numbered vertex to a higher numbered one



## Topological Sort Algorithms

- Algorithm: DFS with a stack ( $O(\#E)$ )
  - Order in which vertices are completed is reverse order of a topological sort!

Stack: 1 2 3 4    Completed: 4

Stack: 1 2 3    Completed: 3

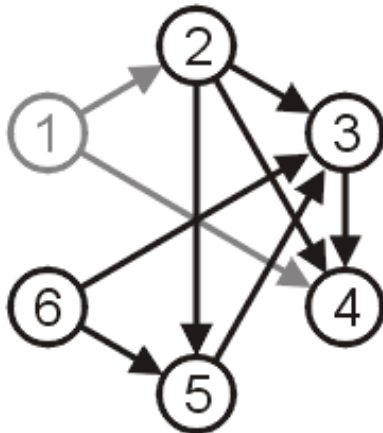
Stack: 1 2 5    Completed: 5

Stack: 1 2    Completed: 2

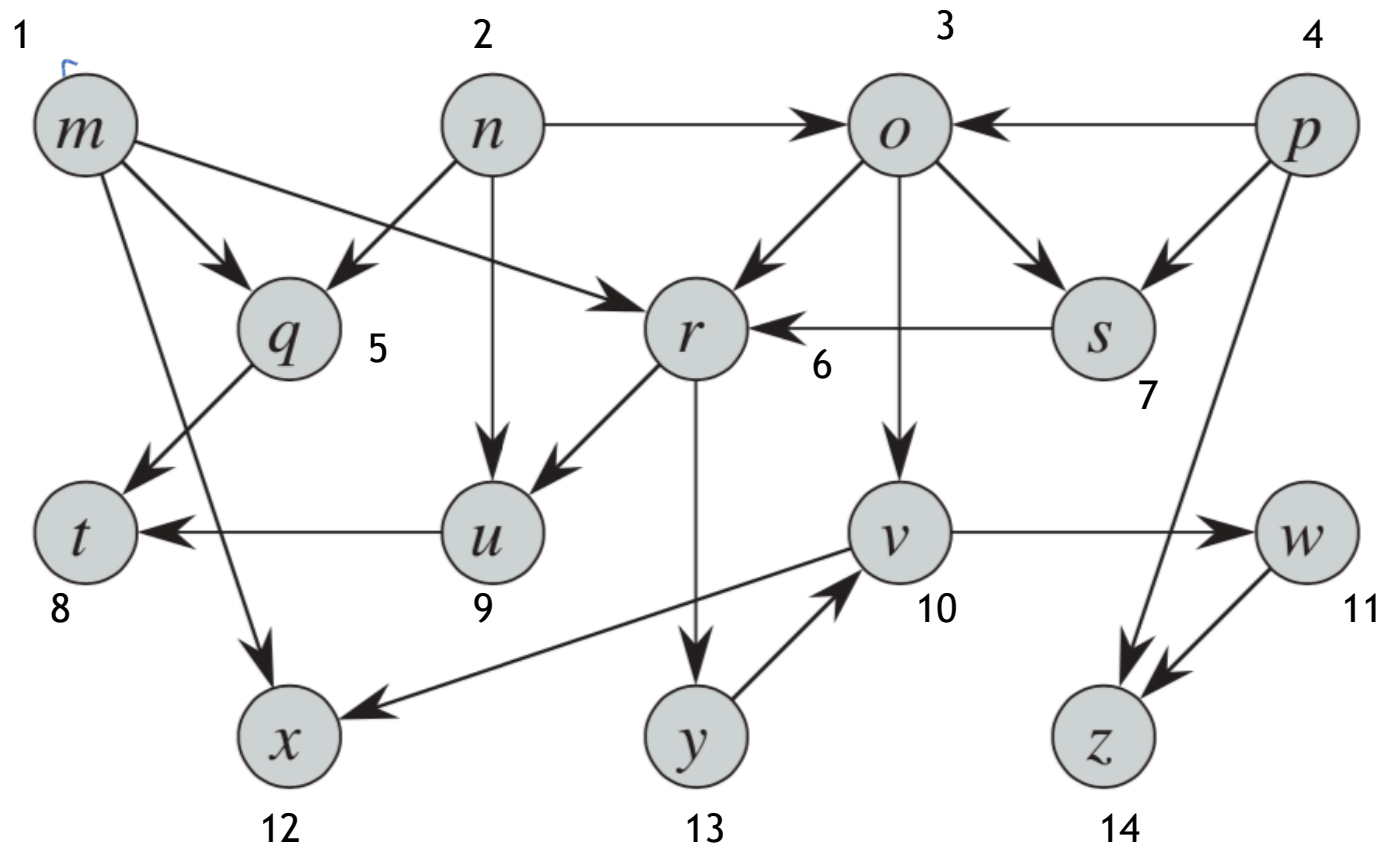
Stack: 1    Completed: 1

Stack: 6    Completed: 6

Reverse Order: 6,1,2,5,3,4



## DAG Topological Sort (Execution order)

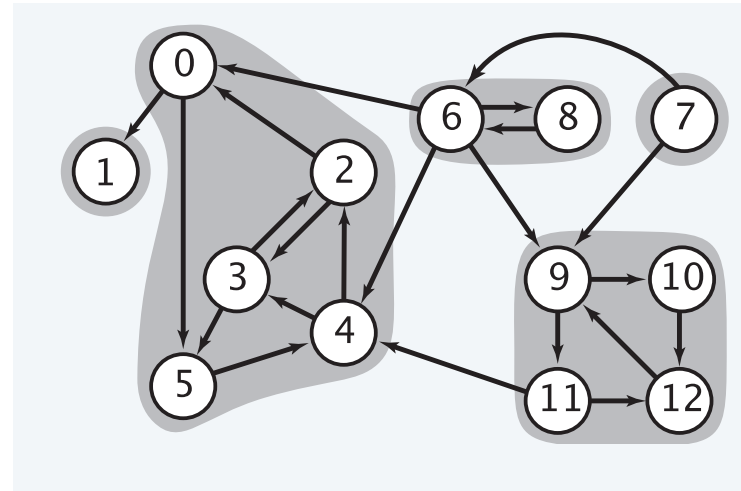


# Strong Connectivity

- A directed graph is strongly connected if every vertex can be reached from every other vertex by a directed path
  - Would like to detect if a graph is strongly connected
- **Lemma.** Let  $s$  be any node.  $G$  is strongly connected iff every node is reachable from  $s$ , and  $s$  is reachable from every node.
- **Algorithm:** Pick any  $s$ .
  - Run BFS from  $s$  and verify all vertices can be reached
  - Reverse all edges in  $E$ , run BFS and verify all vertices can be reached
  - Complexity  $O(\#E + \#V)$  (2 BFS)

# Strongly-Connected Components

- A directed graph is strongly connected if every vertex can be reached from every other vertex
  - A strongly-connected component of a graph is a subgraph that is strongly connected
  - Would like to identify strongly-connected components of a graph
  - Can be used to identify weaknesses in a network
  - General approach: Perform two DFSs

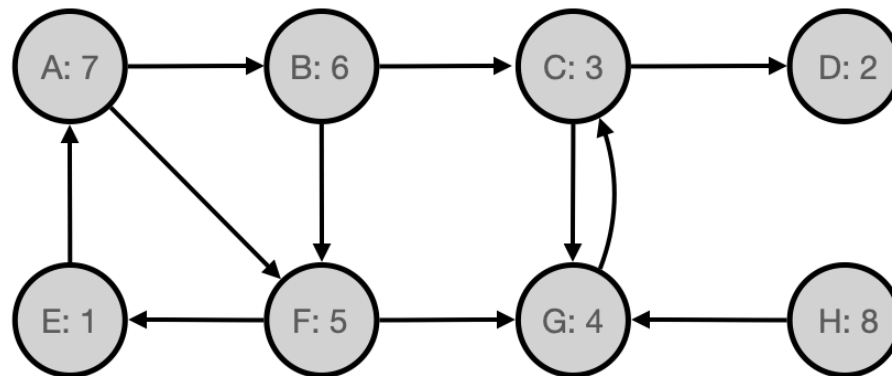
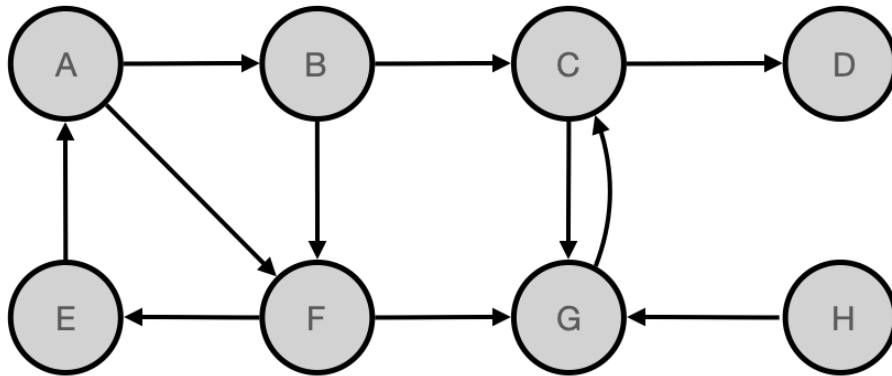


# Strongly-Connected Components

- Kosaraju's Algorithm
  - Perform DFS on directed graph  $G = (V, E)$ ,
    - Number vertices according to their finishing time in DFS of  $G$
  - Perform DFS on  $G_r = (V, E_r)$ , where  $E_r$  are reverse of edges in  $E$ , selecting nodes to start in the stack, in decreasing order of finishing time in previous DFS
  - Strongly connected components = reachable trees obtained in last DFS

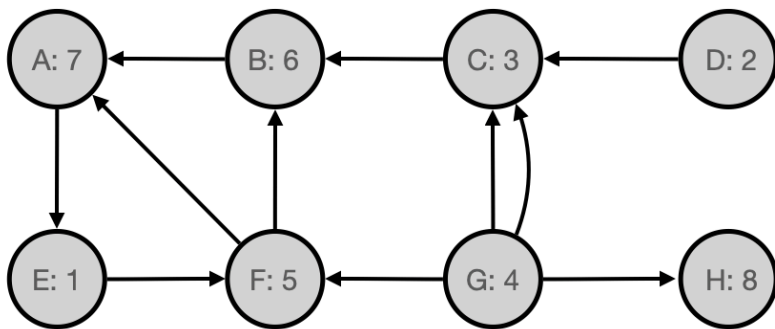


## Example

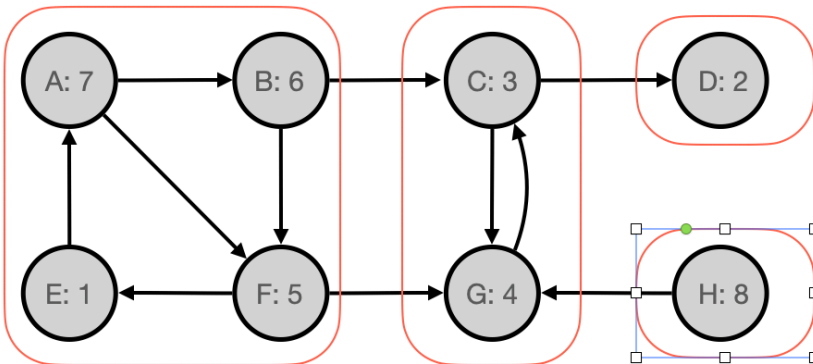


Stack: A B F E: E finishes  
A B F G C D: D finishes  
A B F G C: C finishes  
A B F G: G finishes  
A B F: F finishes  
A B: B finishes  
A: A finishes  
H: H finishes

# Example



Reverse graph with distance labels



Strongly Connected Components

First vertex in stack: H — No out neighbors, pops, it is its own tree, so it is an isolated component

Next vertex in stack: A: adds E, then E adds F, then F adds B. They pop, to form strongly connected component A B E F.

Next highest vertex into stack: G: adds C: Pops C. Strongly connected component G C.

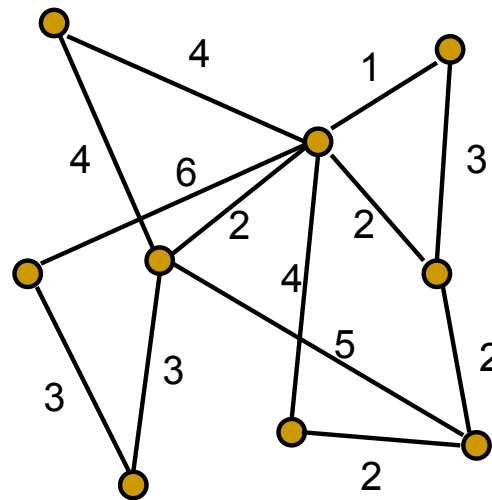
Next highest; D. Add D to stack: pops out already So D is its own connected component.

# Strongly-Connected Components

- Correctness
  - If  $v$  and  $w$  are in a strongly-connected component
  - Then there is a path from  $v$  to  $w$  and a path from  $w$  to  $v$
  - Therefore, there will also be a path between  $v$  and  $w$  in  $G$  and  $G^r$
- Running time
  - Two executions of DFS
  - $O(|E|+|V|)$

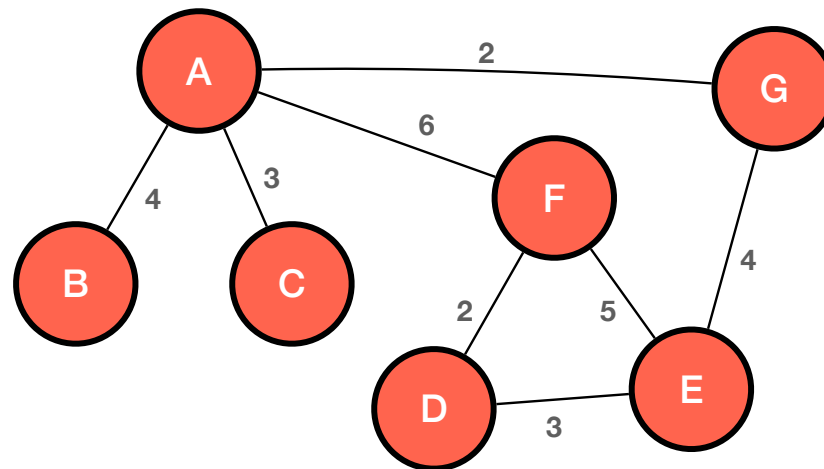
# Weighted Graphs

- A weighted graph  $G = (V, E)$  is a graph along with a weight function  $w : E \rightarrow \mathfrak{R}$
- Weighted graphs can be directed or undirected



# Spanning Trees

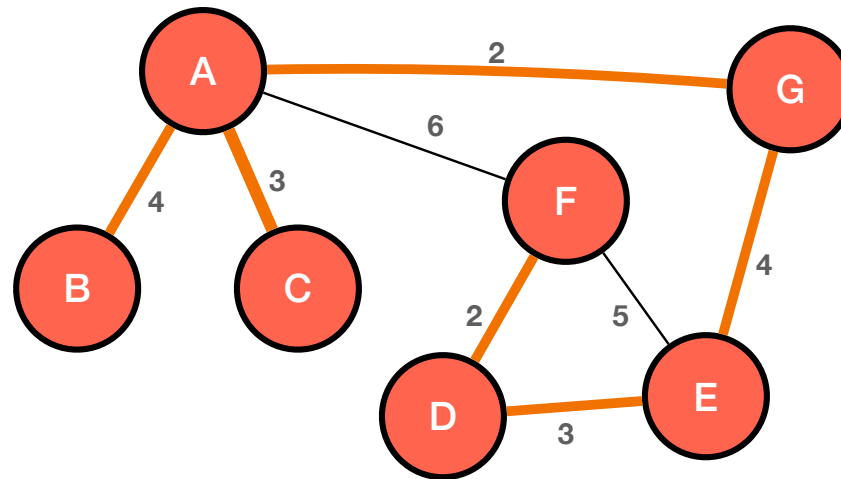
- A **spanning tree** of an undirected graph is
  - edge subset forming a tree that spans every vertex
  - $\#V - 1$  edges



# Minimum Spanning Trees

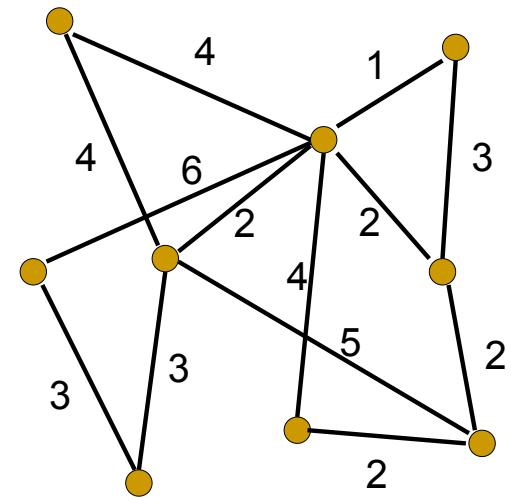
- A **minimum spanning tree** (MST) of an undirected weighted graph  $(V, E)$  with weights  $w(\cdot)$ :

Connected subgraph  $(V, E')$  which is a tree and for which  $\sum_{\{i,j\} \in E'} w(i,j)$  is minimized



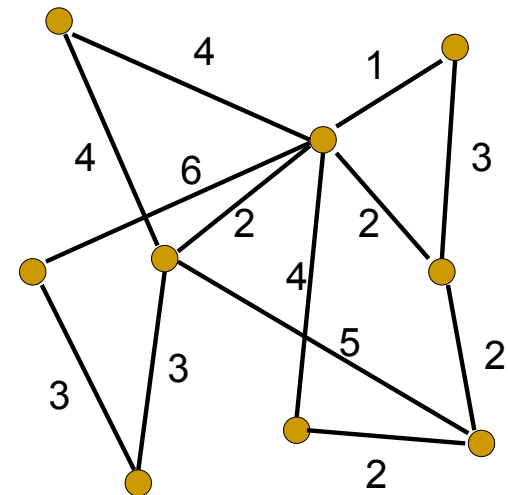
# MST Problem

- Given a weighted graph  $G$ , we want a subgraph  $G' = (V, E')$ ,  $E' \subseteq E$ , such that all vertices are **connected** on  $G'$   
Subgraph  $G' = (V, E')$ ,  $E' \subseteq E$   
total **weight**  $\sum_{(x,y) \in E'} w(x,y)$  is minimized
- Spanning tree**: a tree containing all vertices in  $G$
- Question**: Find a spanning tree with minimum weight.
  - The problem is thus called **Minimum Spanning Tree (MST)**



# MST: Problem and Motivation

- Suppose we have  $n$  computers, connected by wires as given in the graph
- Each wire has a renting **cost**
- We want to select some wires, such that all computers are **connected** (i.e. every two can communicate)
- How to select a subset of wires with the **minimum** renting cost?
- Other applications: Networks
  - Networks: electric, computer, water, transportation
  - Computer vision: Facial recognition, handwriting recognition, image segmentation
  - Low-density parity check codes (LDPC)



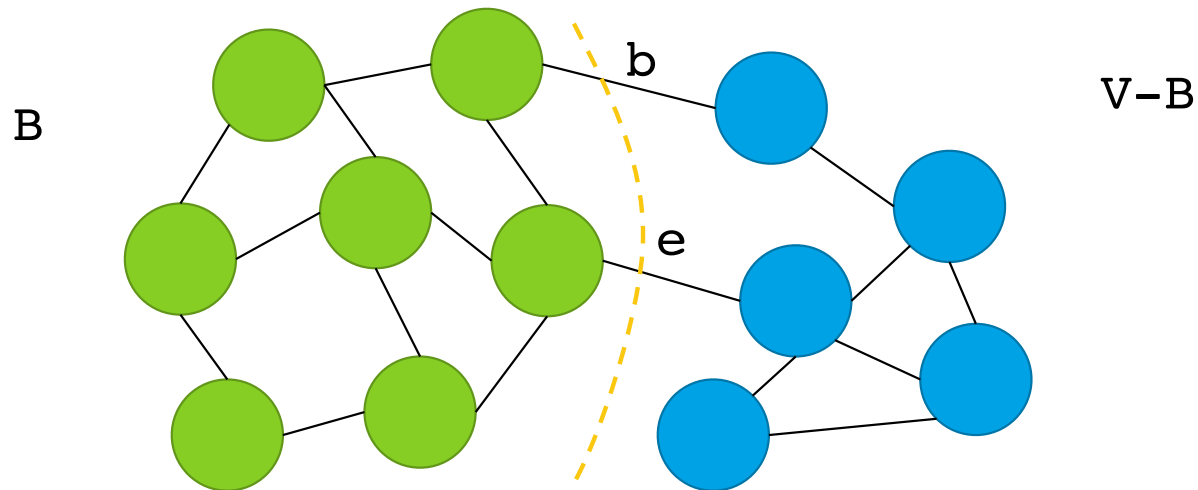


# Preliminary ideas

- Build minimal spanning trees incrementally
- Show by induction that algorithm is correct at each step
- Concept: A set of edges  $T \subset E$  is **promising** if it is a subset of a minimal spanning tree  $(V, E')$ 
  - Let  $B = \{v \in V : v \in e \text{ for some } e \in T\}$
  - Then,  $(B, T)$  is a subgraph of  $(V, E)$  which is a forest (no cycles)

# Graph Cuts

- ▶ A cut is any partition of the vertices into two groups,  $B$  and  $V - B$



- ▶ with edges  $b$  and  $e$  joining the partitions

## Useful Property

- **Lemma:** Let  $T \subset E$ , and  $B \subset V, B \neq V$ . Assume  $T$  is a promising set of edges for the MST problem in  $(V, E)$ , and no edge in  $T$  leaves  $B$  (no edge across cut  $(B, V-B)$ ). Let  $e^*$  be the smallest weight edge in  $E$  such that  $e^* = \{i, j\}, i \in B, j \notin B$ . Then,  $T' = T \cup \{e^*\}$  is a promising set.
- Allows us to grow a promising set! (Induction)

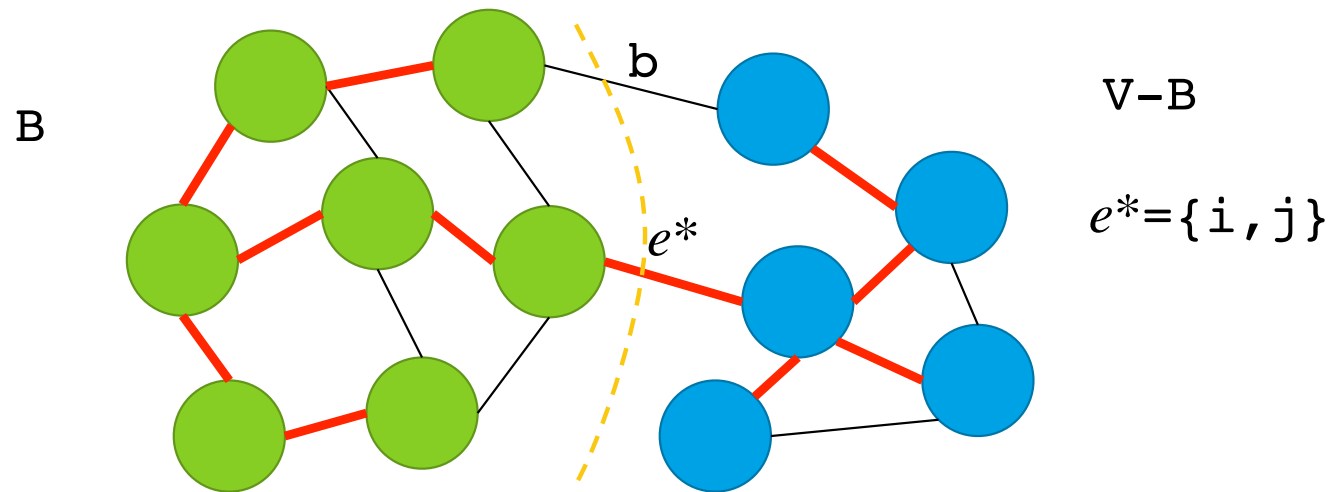
**Proof.**

$(B, V-B)$  form a partition of  $V$ : a cut

Edge  $e^* = \{i, j\}, i \in B, j \notin B$  has the smallest weight among all edges that cross the cut.

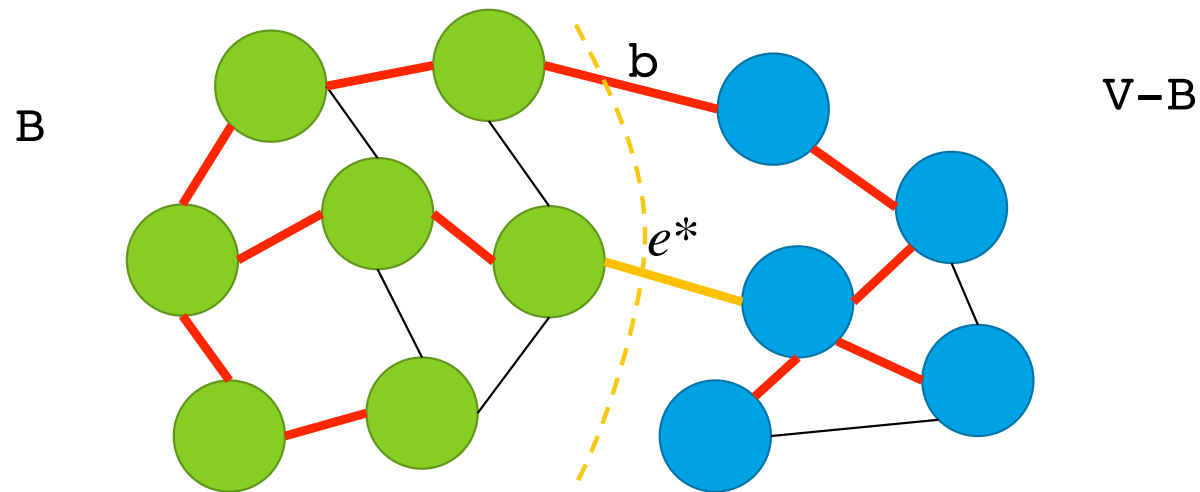
## Case 1: MST includes $e^*$

- ▶ Let MST  $(G, E')$  where  $T \subset E'$  be such that  $e^* \in E'$
- ▶ Simple: then  $T \cup \{e^*\} \subset E'$  is promising



## Case2: MST does not includes e

- ▶ Let MST  $(G, E')$  where  $T \subset E'$  be such that  $e^* \notin E'$
- ▶ Hard: Look at cycle including e. Swap b for  $e^*$  in  $E'$ , total weight must not increase, hence have MST  $(V, E' \cup \{e^*\} - \{b\})$



▶

# Proof of Correctness

▸ **Lemma:** Proof (cont)

Since  $T$  is promising, let  $MT = \text{minimum spanning tree } (V, E'), T \subset E'$ .

If  $e^* \in E'$ , then  $T' = T \cup \{e^*\} \subset E'$  has no cycles, and is thus a promising set of edges, proving the theorem.

If  $e = \{i, j\} \notin E'$ , then  $E' \cup \{e^*\}$  has one cycle that includes  $e^*$ , as it has  $\#V$  edges and a tree must have at most  $\#V-1$  edges

In that cycle, there exists edge  $b$  that leaves  $B$ , hence  $b \notin T$

Note  $w(b) \geq w(e^*)$  by how we selected  $e^*$

Note:  $E' \cup \{e^*\} - \{b\}$  leaves graph connected, and has number of edges =  $\#V-1$ , and its total weight is no greater than the weight of  $E'$

Hence,  $(V, E' \cup \{e^*\} - \{b\})$  is also MST, and  $T' = T \cup \{e^*\} \subset E' \cup \{e^*\} - \{b\}$  so  $T'$  is a promising subset

# Kruskal's Algorithm

- Sort edges by weight in ascending order
- Start with empty set  $T$  (note: it is promising)
- For each edge  $e$  in sorted list
  - If adding edge  $e$  to  $T$  does not create cycle in  $(V, T \cup e)$
  - ...add it to MST:  $T = T \cup \{e\}$
  - Claim:  $T$  is now promising set with one more edge
- Stop when you have  $\#V - 1$  edges in  $T$

# Proof of Correctness

- At any stage in algorithm,  $T$  is a forest (no cycles can be created)
- At any stage in algorithm,  $T$  is a promising set
  - True initially when  $T = \emptyset$
  - If at a stage in algorithm, edge  $e^*$  is the lowest weight edge remaining, and it does not form a cycle with  $T$ , then:
    - Let  $\{a,b\} = e^*$ ; let  $B$  be vertices in  $T$  connected to  $a$
    - Then, no edge in  $T$  leaves  $B$  and  $e^*$  is minimum weight edge among edges that leave  $B$
    - By lemma,  $T \cup \{e^*\}$  is promising set
- Thus, algorithm converges to MST



# Kruskal

- How can we tell if adding edge will create cycle?
- Start by giving each vertex its own “cloud”, which consists of all connected vertices in current  $T$  (Disjoint Sets)
- If both ends of lowest-cost edge are in same cloud
  - we know that adding the edge will create a cycle!
- When edge is added to MST
  - merge clouds of the endpoints

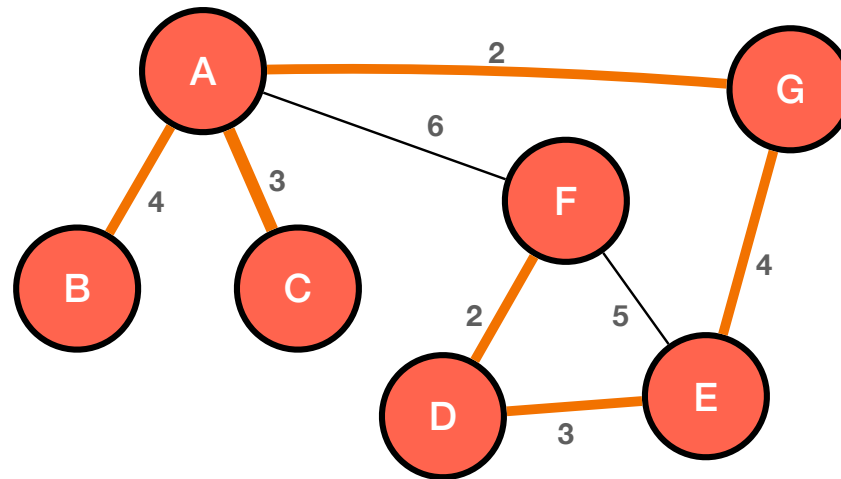
# Kruskal Pseudo-Code

```
function kruskal(G):  
    // Input: undirected, weighted graph G  
    // Output: list of edges in MST  
    for vertices v in G:  
        makeCloud(v) // put every vertex into its own set  
    MST = []  
    Sort all edges  
    for all edges (u,v) in G sorted by weight:  
        if u and v are not in same cloud:  
            add (u,v) to MST  
            if size(MST) = |V| - 1:  
                break  
            merge clouds containing u and v  
    return MST
```

# Minimum Spanning Trees

- A **minimum spanning tree** (MST) of an undirected weighted graph  $(V, E)$  with weights  $w(\cdot)$ :

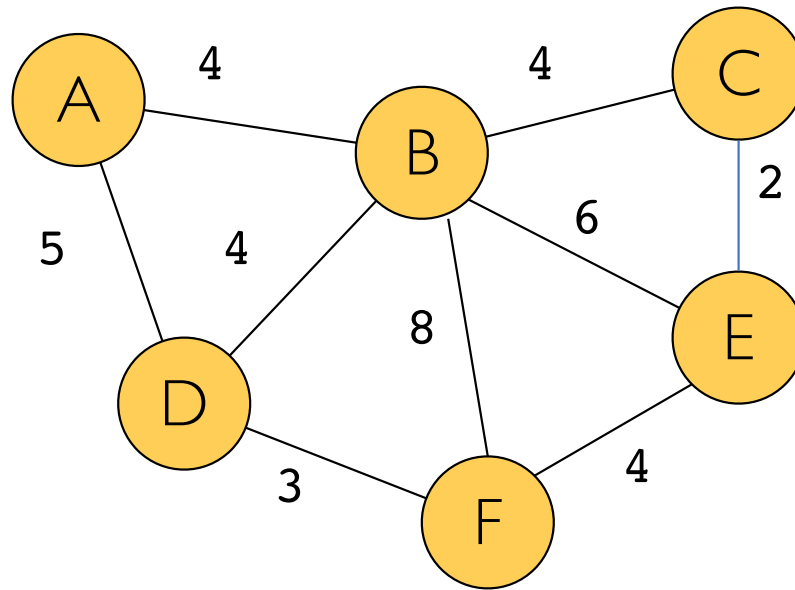
Connected subgraph  $(V, E')$  which is a tree and for which  $\sum_{\{i,j\} \in E'} w(i,j)$  is minimized



# Kruskal's Algorithm

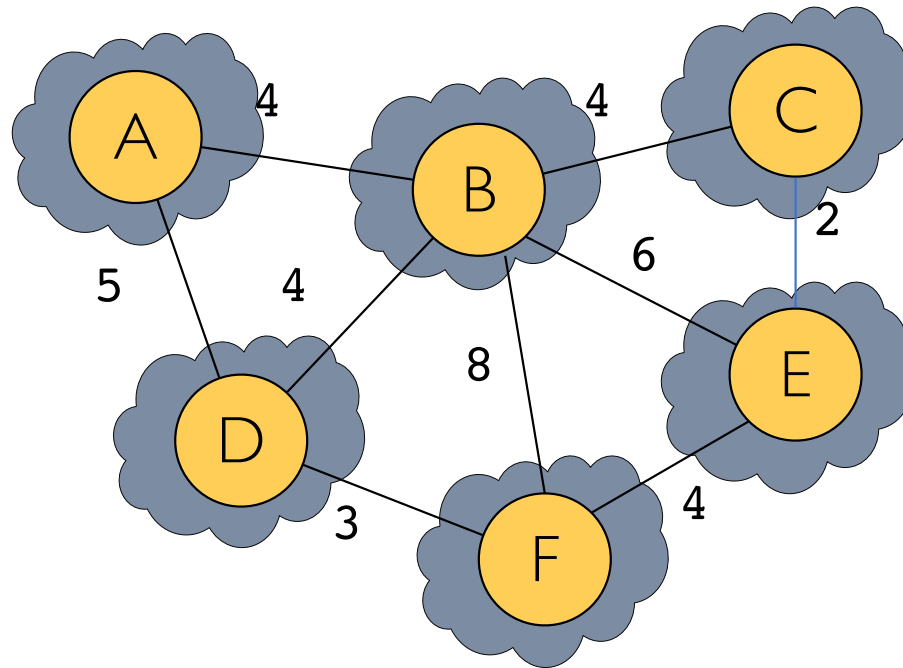
- Sort edges by weight in ascending order
- Start with empty set  $T$  (note: it is promising)
- For each edge  $e$  in sorted list
  - If adding edge  $e$  to  $T$  does not create cycle in  $(V, T \cup e)$
  - ...add it to MST:  $T = T \cup \{e\}$
  - Claim:  $T$  is now promising set with one more edge
- Stop when you have  $\#V - 1$  edges in  $T$

# Example



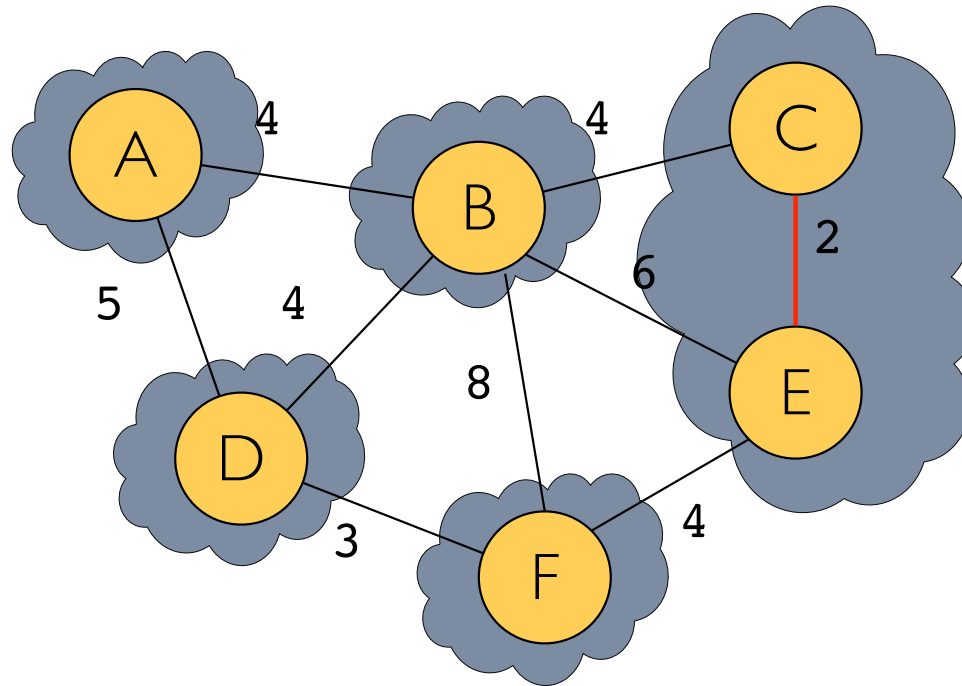
```
edges = [(C,E),(D,F),(B,C),(E,F),(B,D),(A,B),(A,D),(B,E),(B,F)]
```

# Example



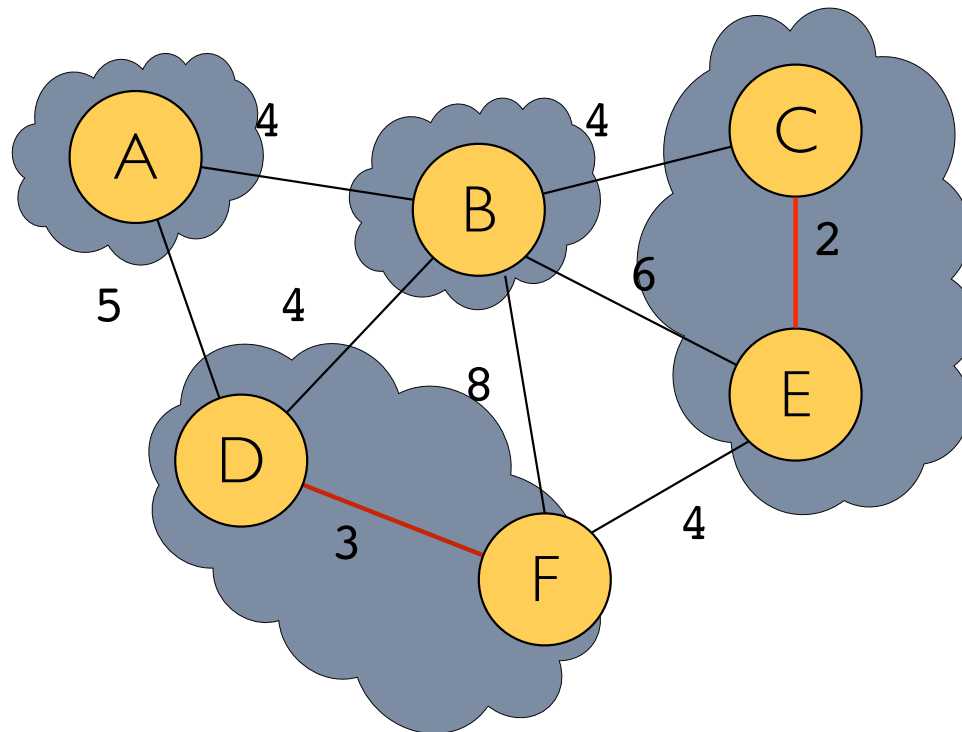
```
edges = [(C,E),(D,F),(B,C),(E,F),(B,D),(A,B),(A,D),(B,E),(B,F)]
```

# Example



```
edges = [ (D,F) , (B,C) , (E,F) , (B,D) , (A,B) , (A,D) , (B,E) , (B,F) ]
```

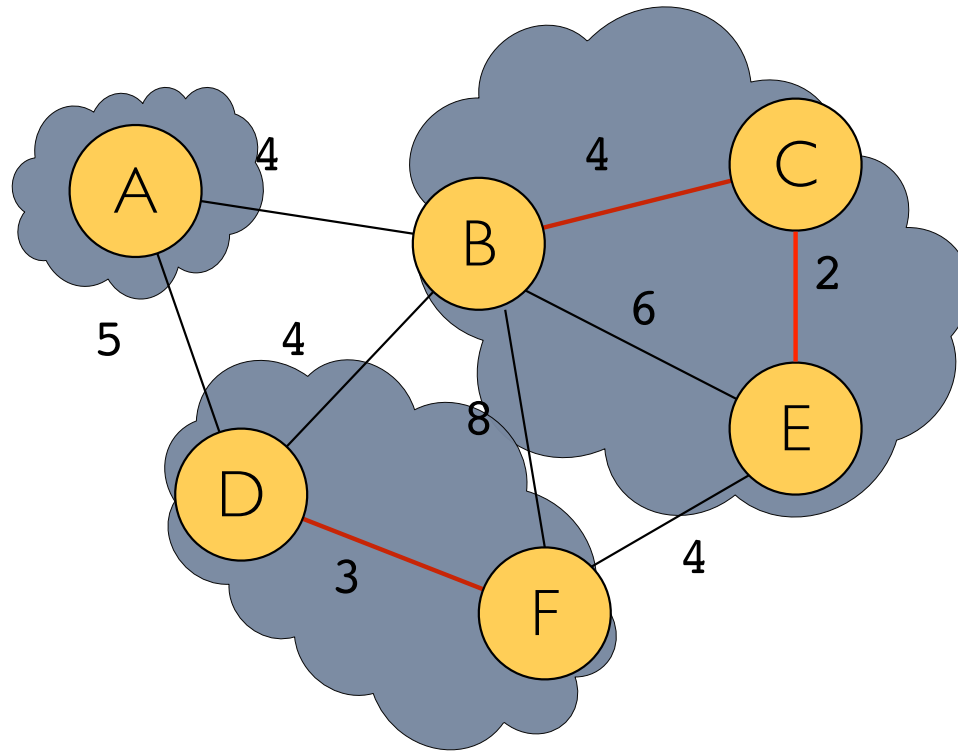
# Example



`edges = [(B,C), (E,F), (B,D), (A,B), (A,D), (B,E), (B,F)]`

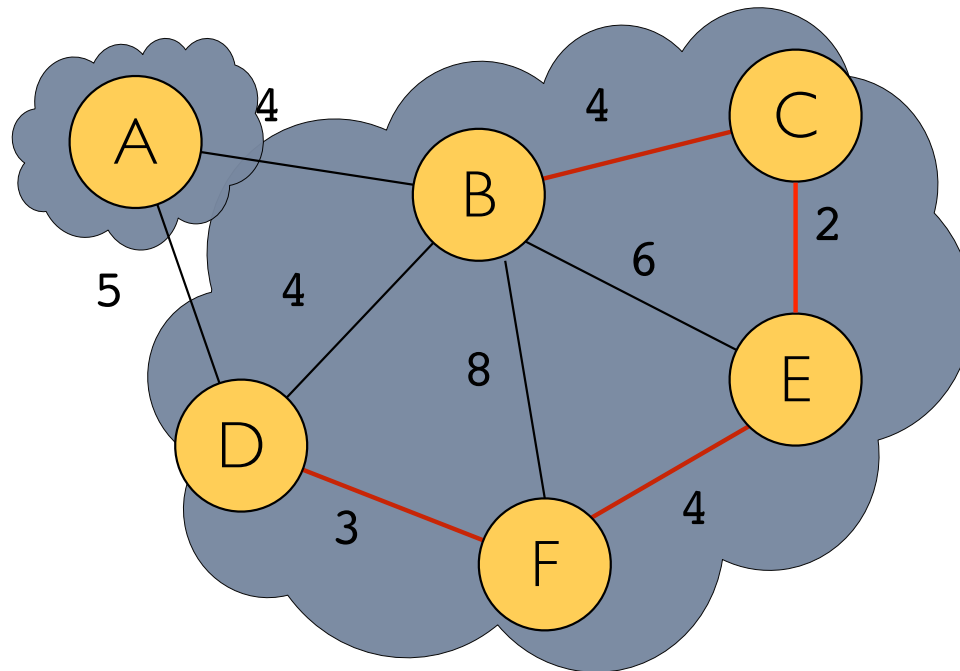


# Example



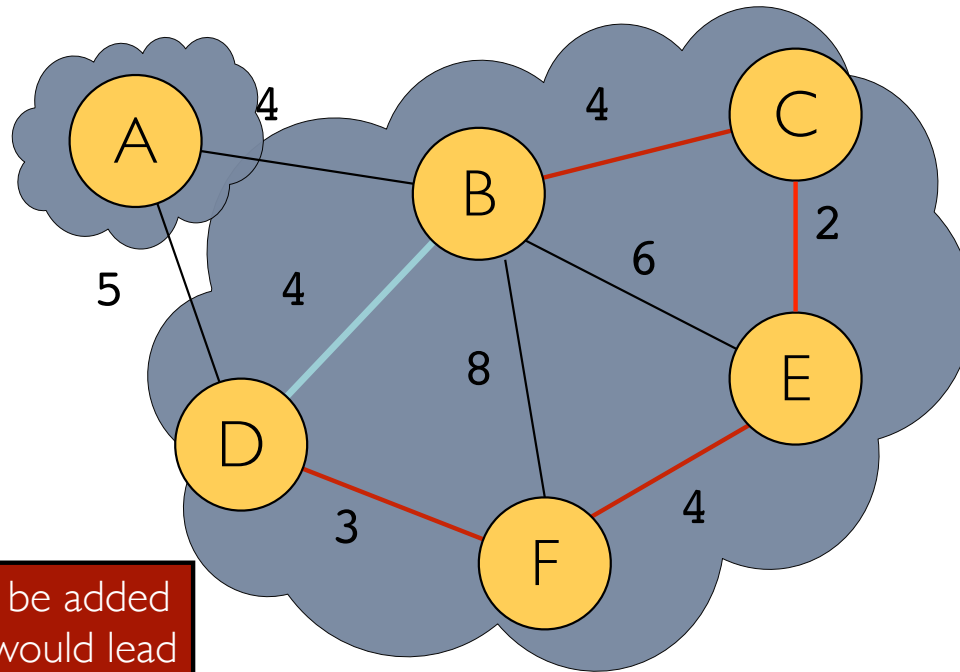
`edges = [ (E,F) , (B,D) , (A,B) , (A,D) , (B,E) , (B,F) ]`

# Example



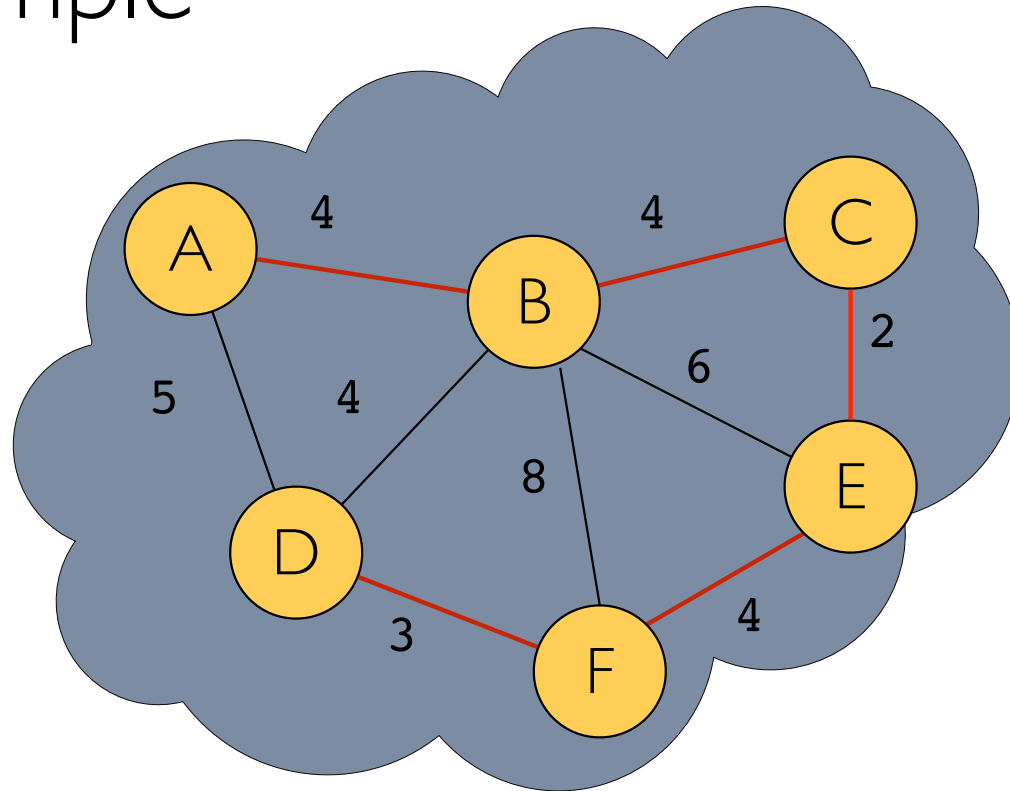
edges = [ (B,D) , (A,B) , (A,D) , (B,E) , (B,F) ]

# Example



edges = [ (A,B) , (A,D) , (B,E) , (B,F) ]

# Example



`edges = [ (A,D) , (B,E) , (B,F) ]`

# Merging Clouds Efficiently (Naive way)

- ▶ Keep track of clouds using disjoint sets (Union-Find)
  - ▶ Edge forms a cycle if both ends of the edge belong to the same disjoint set (have the same root!)
  - ▶ Checking that an edge forms a cycle has amortized complexity  $O(\log^*(|V|)) \approx O(1)$

# Kruskal Runtime

- $O(|V|)$  for iterating through vertices
- $O(|E|\log|E|)$  for sorting edges
- $O(|E|\times 1)$  for iterating through edges and merging clouds with path compression
- $O(|V|+|E|\log|E|+|E|\times 1)$ 
  - $= O(|V|+|E|\log|E|)$
- $O(|V|+|E|\log|E|)$ 
  - Better than simple  $O(|V|^3)$  without disjoint sets

# Example

Disjoint components:

1 2 3 4 5 6 7 8 9

1) Add {3,6}

Disjoint components:

3 1 2 4 5 7 8 9

6

2) Add {4,6}

3 1 2 5 7 8 9

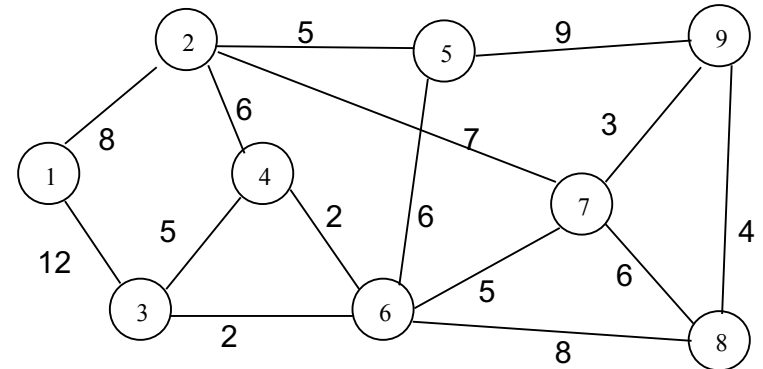
/ \

6 4

Minimum Spanning Tree

Empty

Minimum Spanning Tree



Sorted edges

{3,6}, {4,6}, {7,9}, {8,9}, {3,4}, {6,7},  
 {2,5}, {2,4}, {5,6}, {7,8}, {2,7}, {1,2},  
 {6,8}, {5,9}, {1,3}

# Example

Disjoint components:

3) Add {7,9}

3      7    1 2 5 8

/ \    |

6 4    9

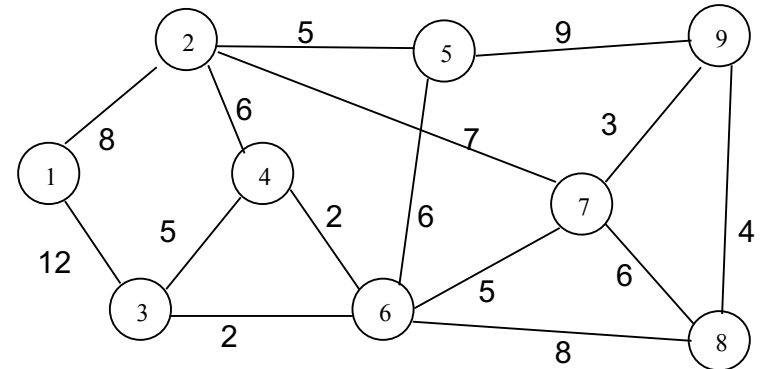
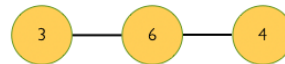
4) Add {8,9}

3          7    1 2 5

/ \      / \

6 4    9 8

Minimum Spanning Tree



Sorted edges

{3,6}, {4,6}, {7,9}, {8,9}, {3,4}, {6,7},  
 {2,5}, {2,4}, {5,6}, {7,8}, {2,7}, {1,2},  
 {6,8}, {5,9}, {1,3}



# Example

Disjoint components:

5) Discard  $\{3,4\}$  (cycle); add  $\{6,7\}$

```

  3      1 2 5
 / | \
6 4 7
   / \
   9 8

```

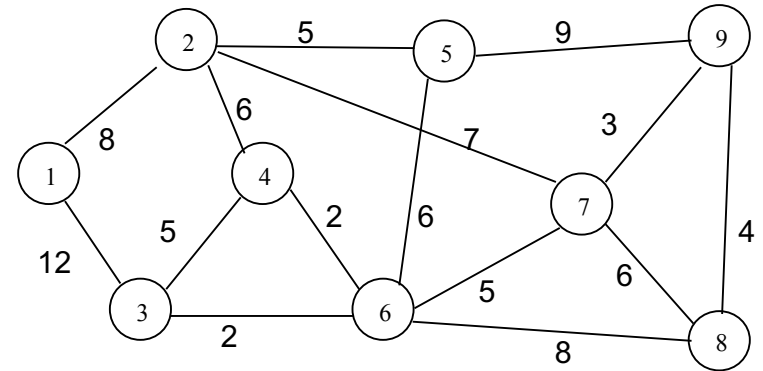
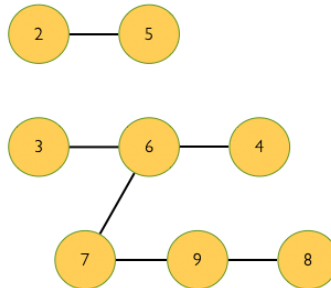
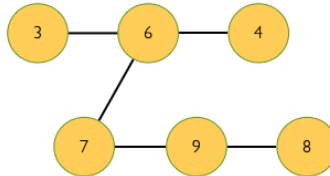
6) Add  $\{2,5\}$

```

  3      1 2
 / | \      \
6 4 7          5
   / \
   9 8

```

Minimum Spanning Tree



Sorted edges

$\{3,6\}, \{4,6\}, \{7,9\}, \{8,9\}, \{3,4\}, \{6,7\},$   
 $\{2,5\}, \{2,4\}, \{5,6\}, \{7,8\}, \{2,7\}, \{1,2\},$   
 $\{6,8\}, \{5,9\}, \{1,3\}$

# Example

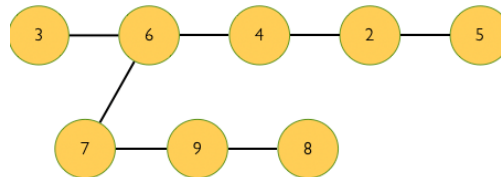
Disjoint components:

7) Add {2,4}

```

( 3 )      1
 / | \ \
6 4 7 2
   / \ \
   9 8 5
    
```

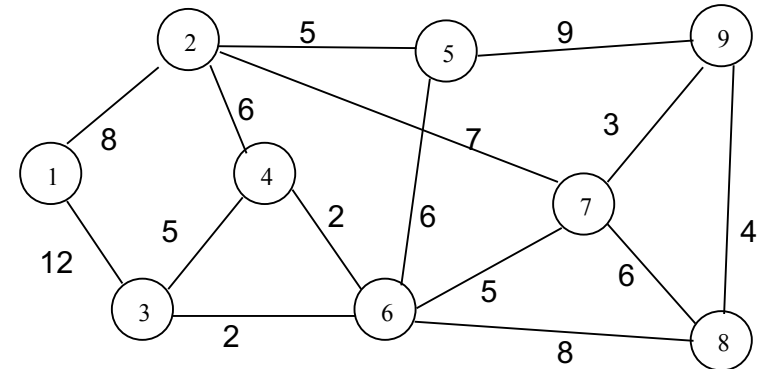
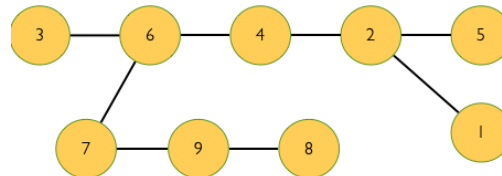
Minimum Spanning Tree



8) Discard {5,6},{7,8}, {2,7}. Select arc {1,2} . DONE

```

( 3 )
 / | \ \ \
6 4 7 2 1
   / \ \
   9 8 5
    
```



Sorted edges

{3,6}, {4,6}, {7,9}, {8,9}, {3,4}, {6,7},  
 {2,5}, {2,4}, {5,6}, {7,8}, {2,7}, {1,2},  
 {6,8}, {5,9}, {1,3}

## Different MST Algorithm: Prim-Jarnik

- Traverse  $G = (V, E)$  starting at any node
  - Maintain priority queue of nodes (e.g. binary heap, Fibonacci heap)
  - set priority to weight of the cheapest edge that connects them to MST
- Un-added nodes start with priority  $\infty$
- At each step
  - Add the node with lowest cost to MST
  - Update (“relax”) neighbors as necessary
- Stop when all nodes added to MST

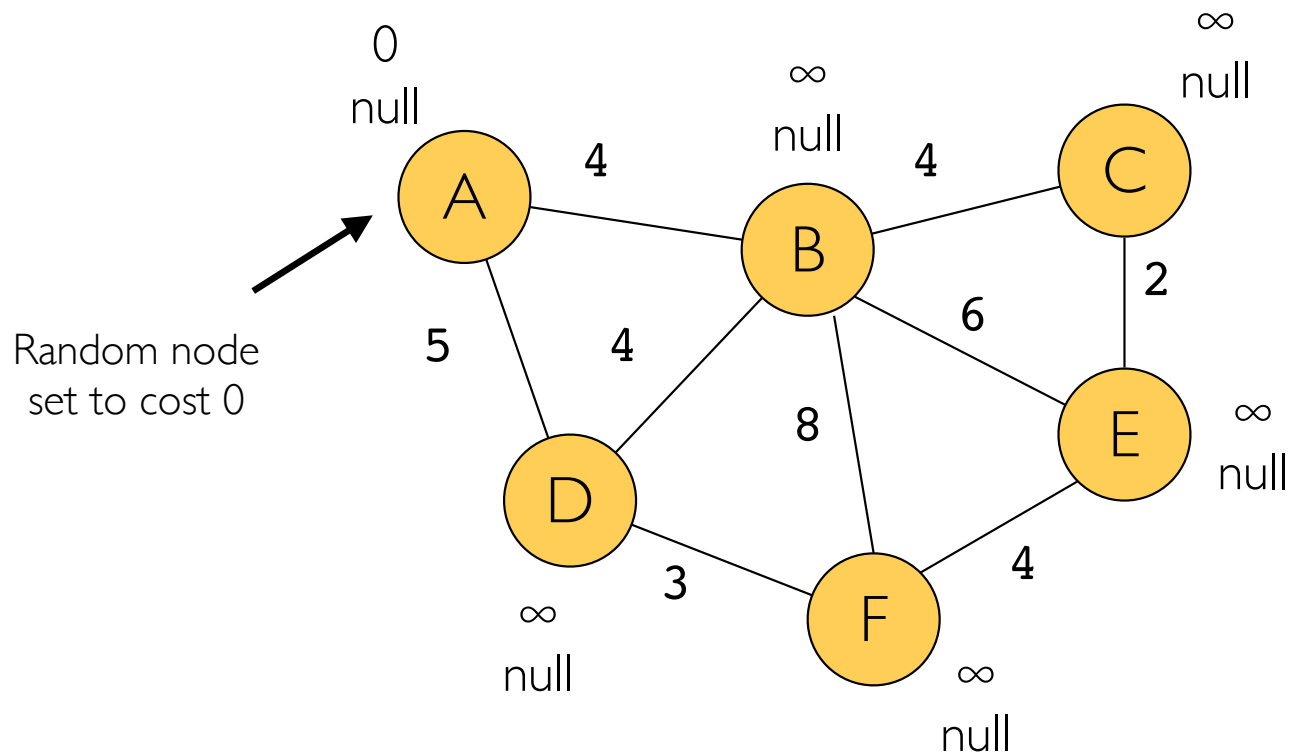
# Different MST Algorithm: Prim-Jarnik

- Traverse  $G = (V, E)$  starting at any vertex  $v$
- Form priority queue PQ of nodes (e.g. binary heap, Fibonacci heap)
  - Set  $v.\text{dist} = 0$ ,  $v'.\text{dist} = \infty$ ,  $v'$  different from  $v$ .  $\text{dist}$  will be the key used in the priority queue
  - Set parent of all vertices,  $v.\text{pred}$  to NULL; set  $T = \emptyset$
- At each step:
  - Remove min from priority queue: call it  $v$ . If  $v.\text{pred}$  not null, add  $\{v, v.\text{pred}\}$  to  $T$
  - For all neighbors  $v'$  of  $v$ : if  $v'$  is in PQ and  $w(v, v') < v'.\text{dist}$ :
    - Decrease  $v'.\text{dist}$  to  $w(v, v')$ , update  $v'.\text{pred} = v$
- Stop when PQ is empty

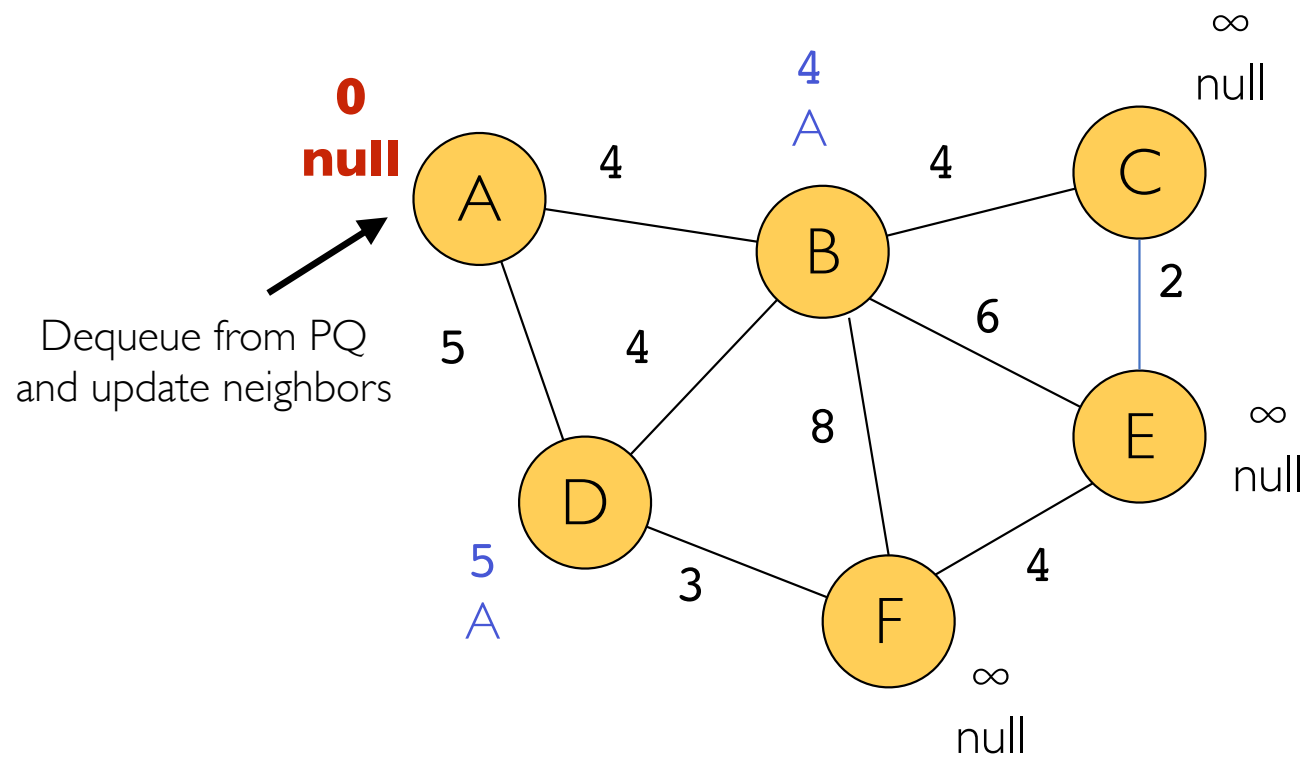
# Pseudo-code

```
function prim(G):  
    // Input: weighted, undirected graph G with vertices V  
    // Output: list of edges in MST  
    for all v in V:  
        v.cost =  $\infty$   
        v.prev = null  
    s = a random v in V // pick a random source s  
    s.cost = 0  
    MST = []  
    PQ = PriorityQueue(V) // priorities will be v.cost values  
    while PQ is not empty:  
        v = PQ.removeMin()  
        if v.prev != null:  
            MST.append((v, v.prev))  
        for all incident edges (v,u) of v such that u is in PQ:  
            if u.cost > (v,u).weight:  
                u.cost = (v,u).weight  
                u.prev = v  
                PQ.decreaseKey(u, u.cost)  
    return MST
```

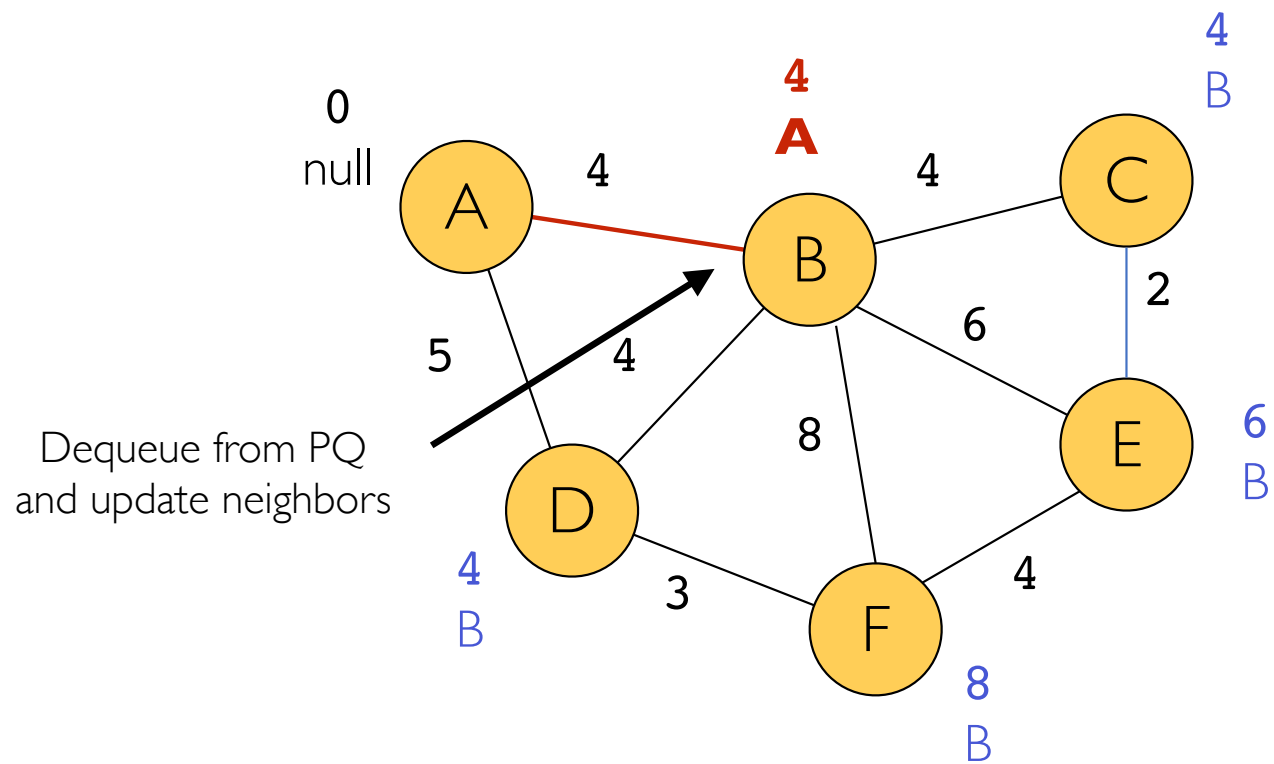
# Example



$PQ = [ (0, A), (\infty, B), (\infty, C), (\infty, D), (\infty, E), (\infty, F) ]$

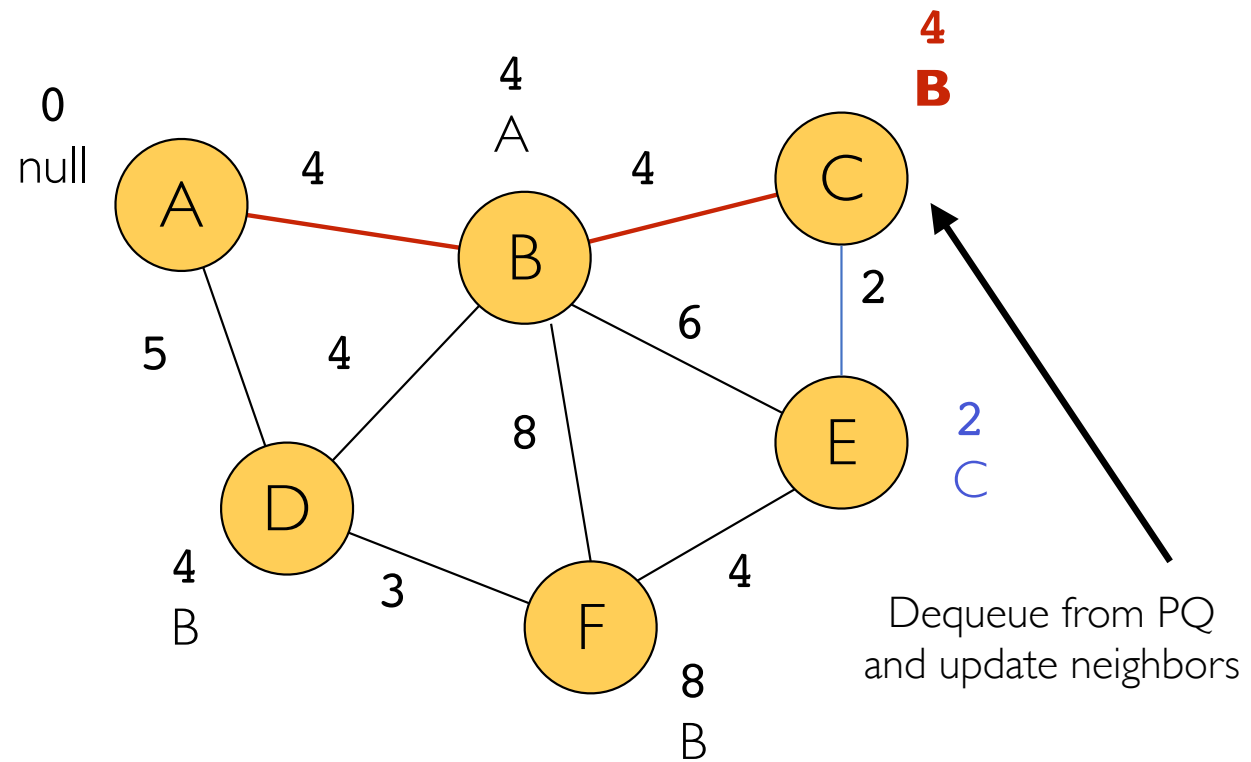


PQ = [ (4, B), (5, D), (∞, C), (∞, E), (∞, F) ]

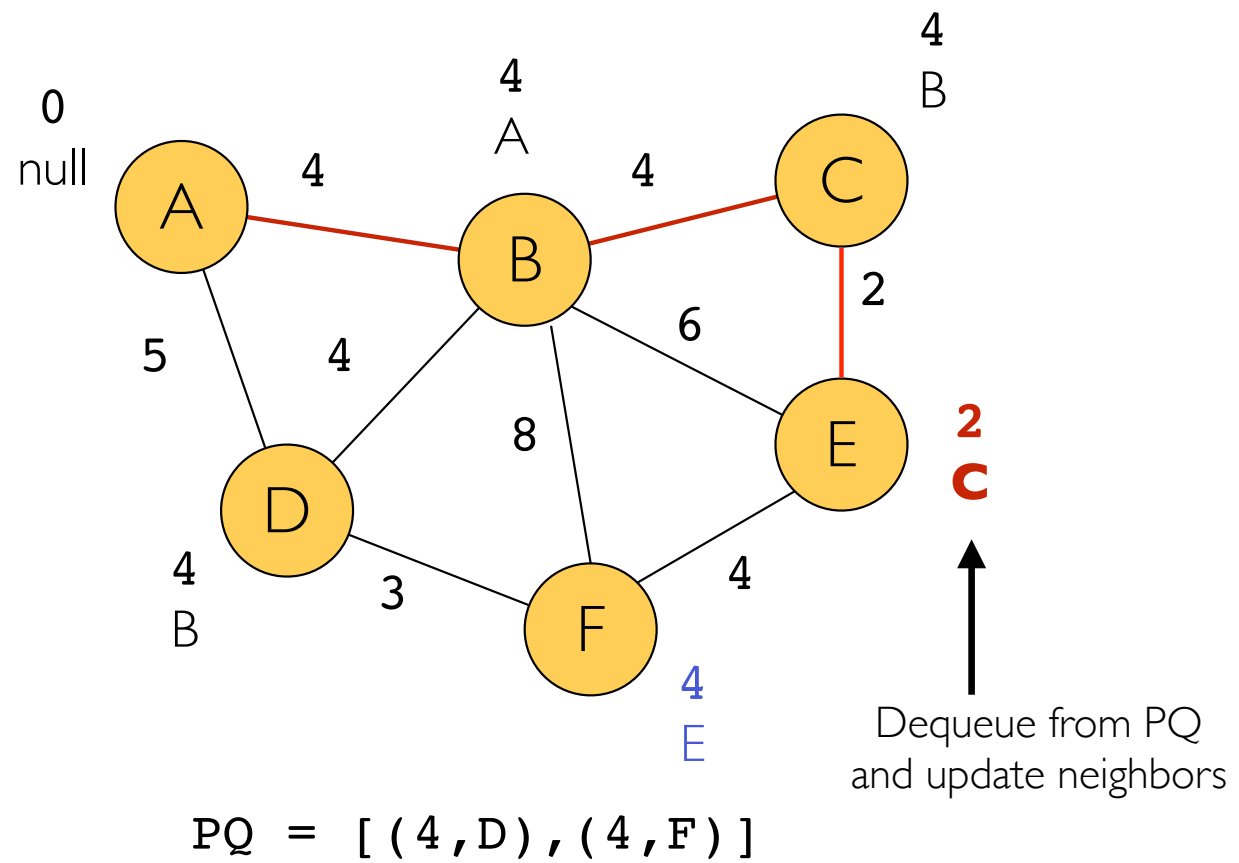


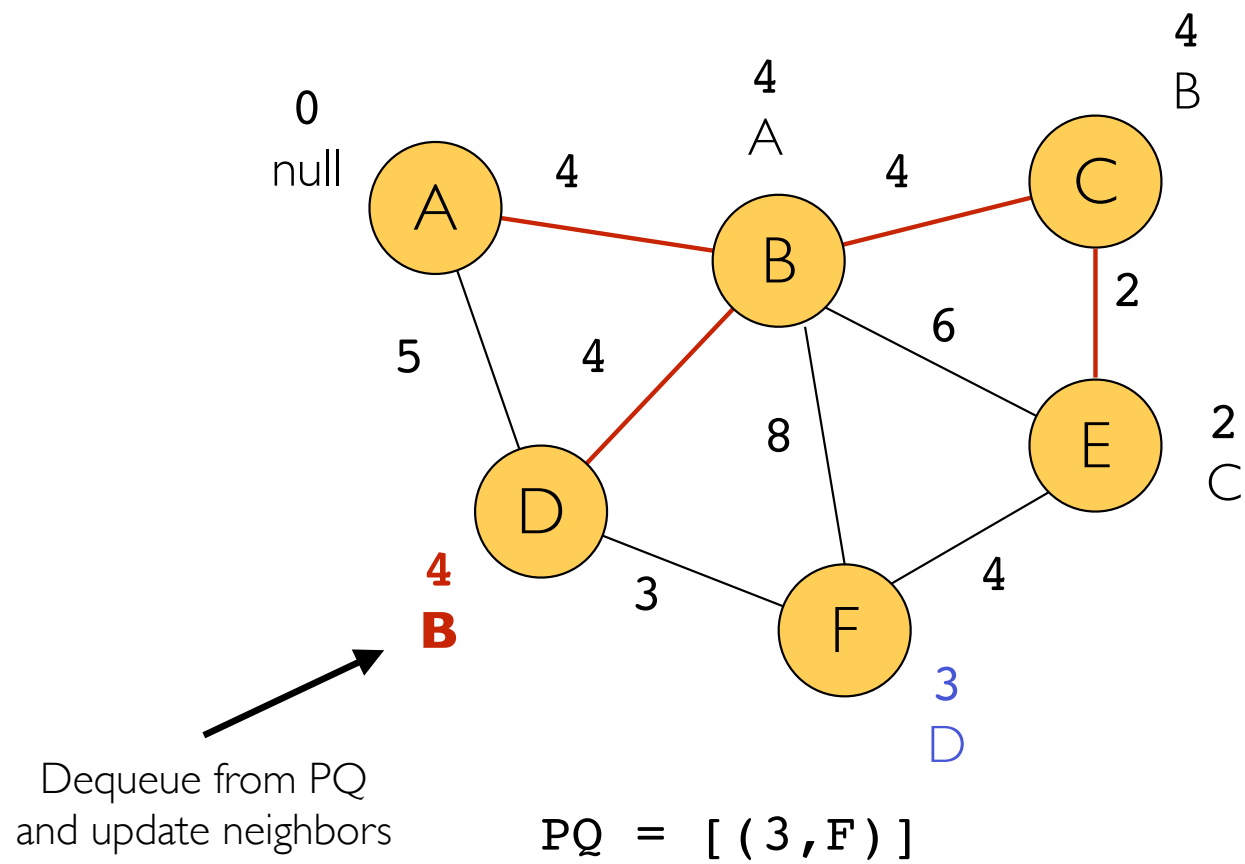
PQ = [ (4, C), (4, D), (6, E), (8, F) ]

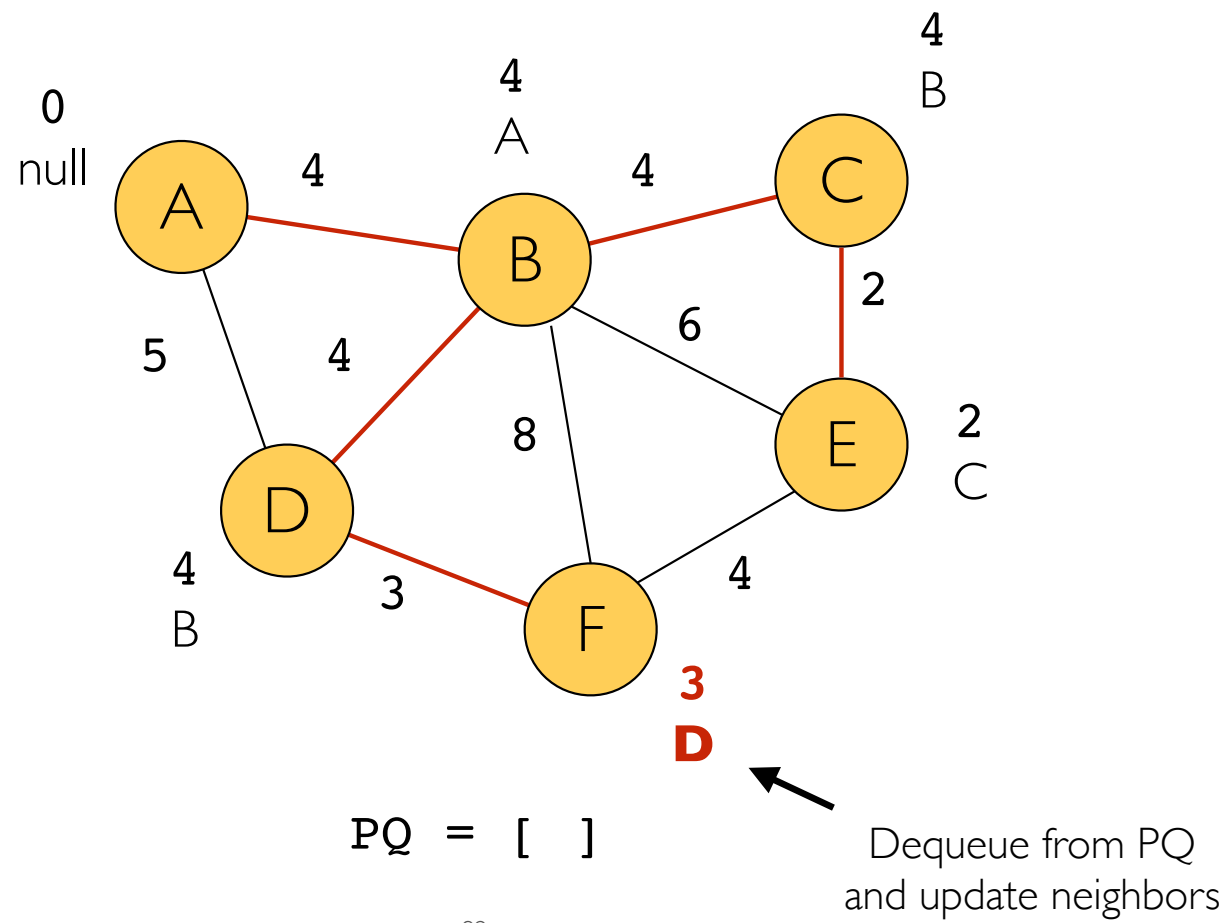


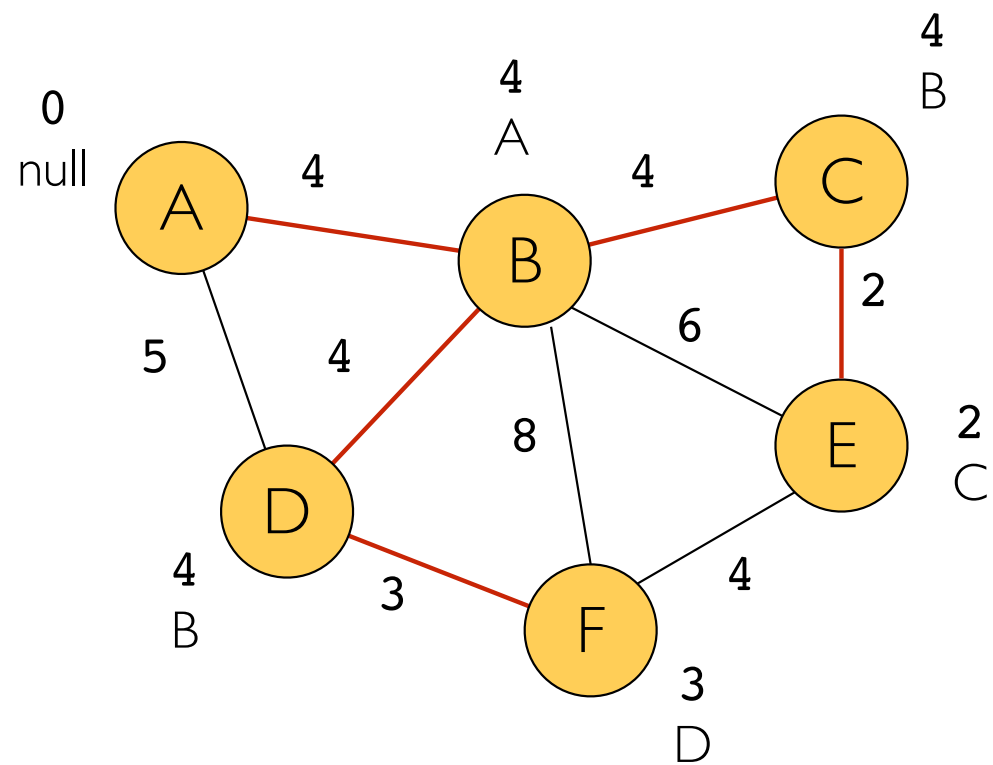


PQ = [ (2, E), (4, D), (8, F) ]









# Proof of Correctness

- Let  $T$  be current tree at iteration step
- Claim:  $T$  is promising set
  - True initially, as  $T = \emptyset$
  - Define  $B =$  vertices in  $T$ : at any stage,  $T$  is a tree over a subset of nodes  $B \subset V$
  - Let  $v$  be min vertex in  $PQ$ ; then  $\{v.\text{parent}, v\}$  leaves  $B$ 
    - $v$  has parent in  $B$ , and  $v$  is still in  $PQ$ , so it is not in  $B$
    - If  $v$  in  $PQ$ ,  $v.\text{dist}$  is set by min weight edge leaving  $B$
  - Thus,  $\{v.\text{pred}, v\}$  is min-weight edge leaving  $B$
  - Hence,  $T \cup \{v.\text{pred}, v\}$  is promising set after remove min extracts  $v$

# Runtime Analysis

- Initializing nodes with distance and previous pointers is  $O(|V|)$ ; putting nodes in PQ is  $O(|V|)$
- While loop runs  $|V|$  times
  - removing vertex from PQ is  $O(\log|V|)$
  - So  $O(|V|\log|V|)$
- For loop (in while loop) runs  $|E|$  times in total
  - Determining whether  $v'$  is in PQ:  $O(1)$  if we build index into PQ (need to find location; not easy!)
  - Decreasing vertex's key in the PQ is  $\log|V|$  (binary heap), or amortized to  $O(1)$  if we use Fibonacci or rank-pairing heaps
  - So  $O(|E|)$  in complex data or  $O(|E| \log|V|)$
- Overall runtime
  - $O(|V| + |V|\log|V| + |E|)$ 
    - $= O(|E| + |V|\log|V|)$  best

# Example

Priority Q

(1,0)

(2,M) (3,M)

(4,M) (5,M) (6,M) (7,M)

(8,M) (9,M)

1) Scan 1, reduce key of 2, 3; remove 1

(2,8)

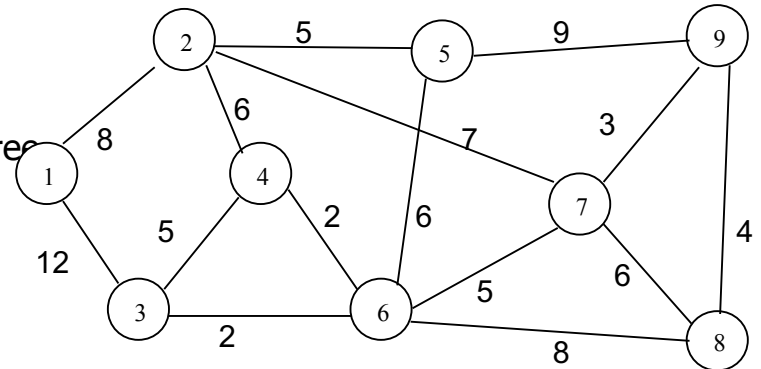
(9,M) (3,12)

(4,M) (5,M) (6,M) (7,M)

(8,M)

Minimum Spanning Tree

Empty



Sorted edges

{3,6}, {4,6}, {7,9}, {8,9}, {3,4}, {6,7},  
 {2,5}, {2,4}, {5,6}, {7,8}, {2,7}, {1,2},  
 {6,8}, {5,9}, {1,3}

1



# Example

2) Scan 2, reduce key of 4,5,7. Remove 2.

Priority Q

(5,5)

(4,6) (7,7)

(9,M) (8,M) (6,M) (3,12)

3) Scan 5, reduce key of 6, 9; remove 5

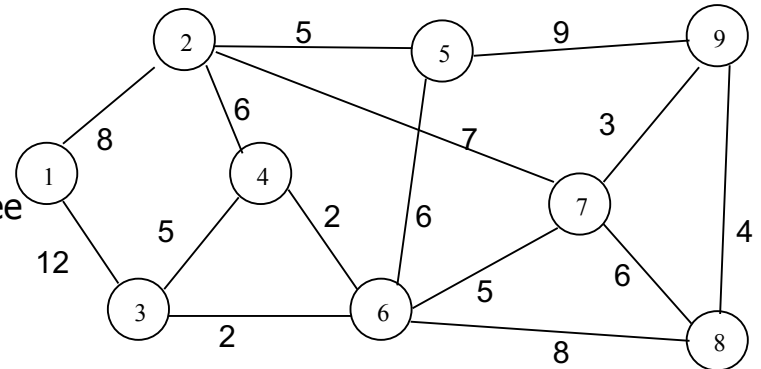
(4,6)

(9,9) (6,6)

(3,12) (8,M) (7,7)

Minimum Spanning Tree

1—2



Sorted edges

{3,6}, {4,6}, {7,9}, {8,9}, {3,4}, {6,7},  
 {2,5}, {2,4}, {5,6}, {7,8}, {2,7}, {1,2},  
 {6,8}, {5,9}, {1,3}



# Example

4) Remove 4, reduce key of 3,6.

Priority Q

(6,2)

(3,5) (7,7)

(9,9) (8,M)

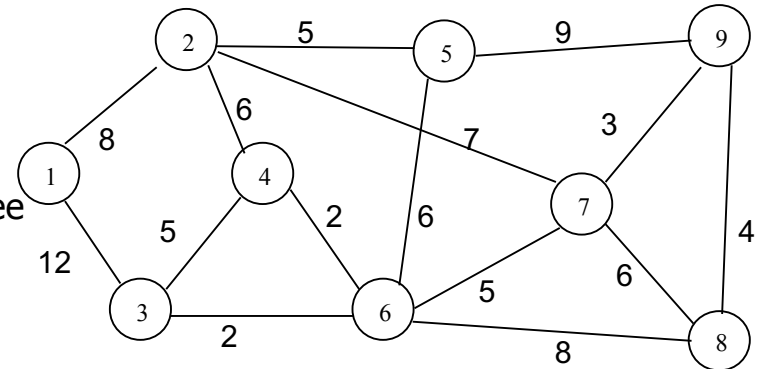
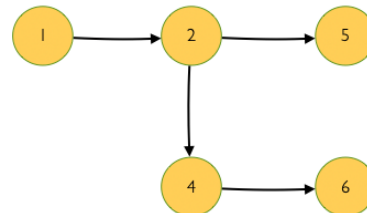
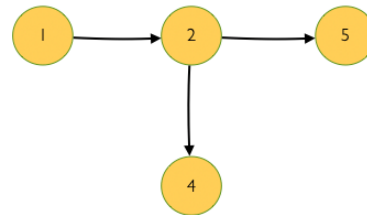
5) Remove 6, reduce key of 3, 7, 8;

(3,2)

(9,9) (7,5)

(8,8)

Minimum Spanning Tree



Sorted edges

{3,6}, {4,6}, {7,9}, {8,9}, {3,4}, {6,7},  
{2,5}, {2,4}, {5,6}, {7,8}, {2,7}, {1,2},  
{6,8}, {5,9}, {1,3}

# Example

6) Remove 3.

Priority Q

(7,5)

(9,9)

(8,8)

5) Remove 7, reduce key of 8,9;

(9,3)

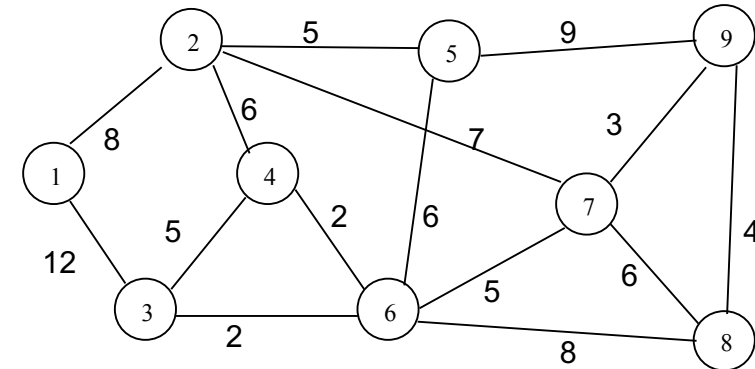
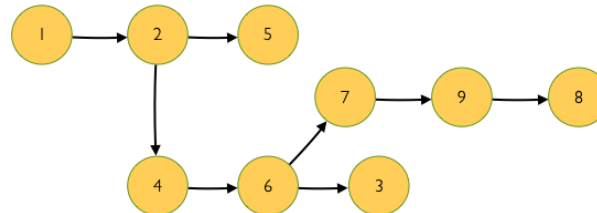
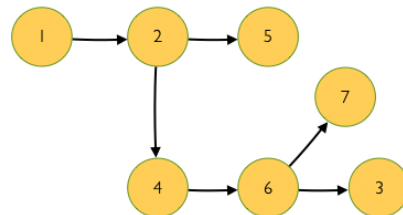
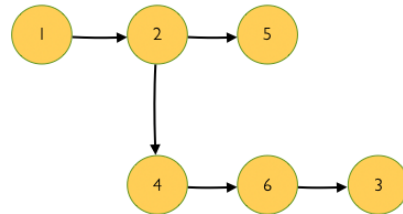
(8,6)

6) Remove 9, reduce key of 8

(8,4)

7) Remove 8. DONE

Minimum Spanning Tree



Sorted edges

{3,6}, {4,6}, {7,9}, {8,9}, {3,4}, {6,7},  
 {2,5}, {2,4}, {5,6}, {7,8}, {2,7}, {1,2},  
 {6,8}, {5,9}, {1,3}

# Summary

- Initializing nodes with distance and previous pointers is  $O(|V|)$ ; putting nodes in PQ is  $O(|V|)$
- While loop runs  $|V|$  times
  - removing vertex from PQ is  $O(\log|V|)$
  - So  $O(|V|\log|V|)$
- For loop (in while loop) runs  $|E|$  times in total
  - Determining whether  $v'$  is in PQ:  $O(1)$  if we build index into PQ (need to find location; not easy!)
  - Decreasing vertex's key in the PQ is  $\log|V|$  (binary heap), or amortized to  $O(1)$  if we use Fibonacci or rank-pairing heaps
  - So  $O(|E|)$  in complex data or  $O(|E| \log|V|)$
- Overall runtime
  - $O(|V| + |V|\log|V| + |E|)$ 
    - $= O(|E| + |V|\log|V|)$  best

## Interesting Facts

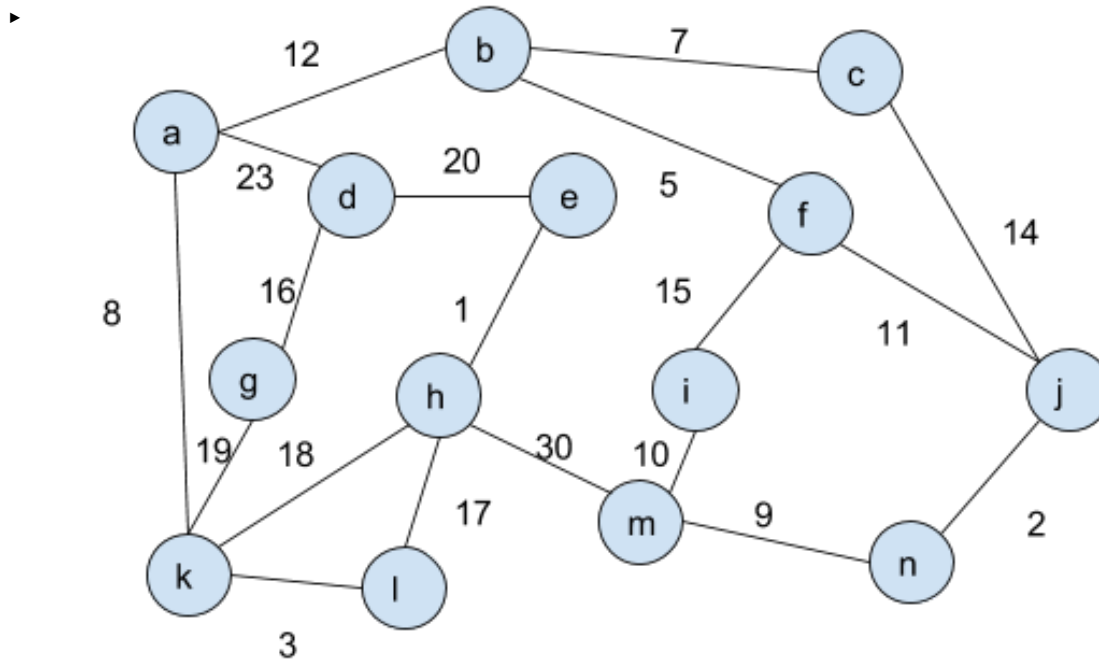
- Cayley's theorem: The complete graph on  $n$  nodes has  $n^{n-2}$  spanning trees
- Suppose that you change the cost of every edge in  $G$  as follows. For which is every MST in  $G$  an MST in  $G'$  (and vice versa)?
  - $c'(e) = c(e) + 17$ .
  - $c'(e) = 17 \times c(e)$ .
  - Assume  $c(e) > 0$ ; then,  $c'(e) = \log_{17} c(e)$
  - All of the above.
- The highest weight edge in every cycle cannot be part of an MST
  - Reverse algorithm: throw out highest weight edges in cycles

# Borůvka's Algorithm

- Earliest MST algorithm: 1926. Application: design of power grid
- For **every** connected set in a forest,
  - Select smallest weight edge that leaves connected set
  - Add it to the MST
  - But don't add it twice if same edge selected by two connected sets
- In principle, merges at least half of the trees at each time:  $O(\log(n))$  iterations
  - Easy to parallelize
- Each pass is  $O(\#E)$

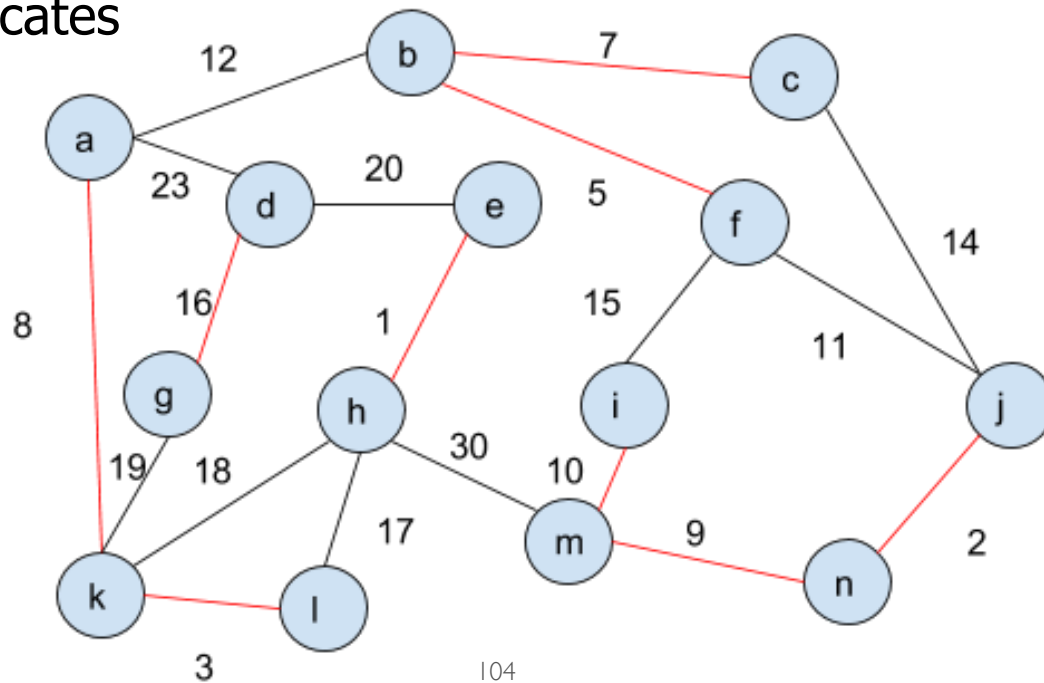
# Example

- Start with every vertex in a separate connected set, partial MST empty



# Example

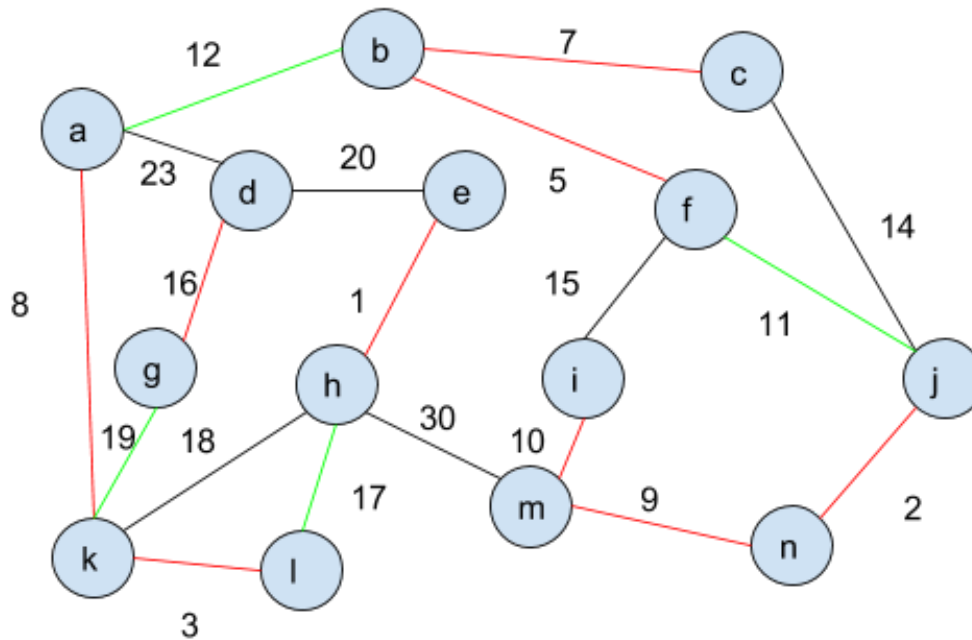
- Find smallest weight edge connected to each set
  - Add all these edges to the partial MST
  - Avoid duplicates





# Example

- Find smallest weight edge connected to each connected set
  - Add all these edges to the partial MST: Done!

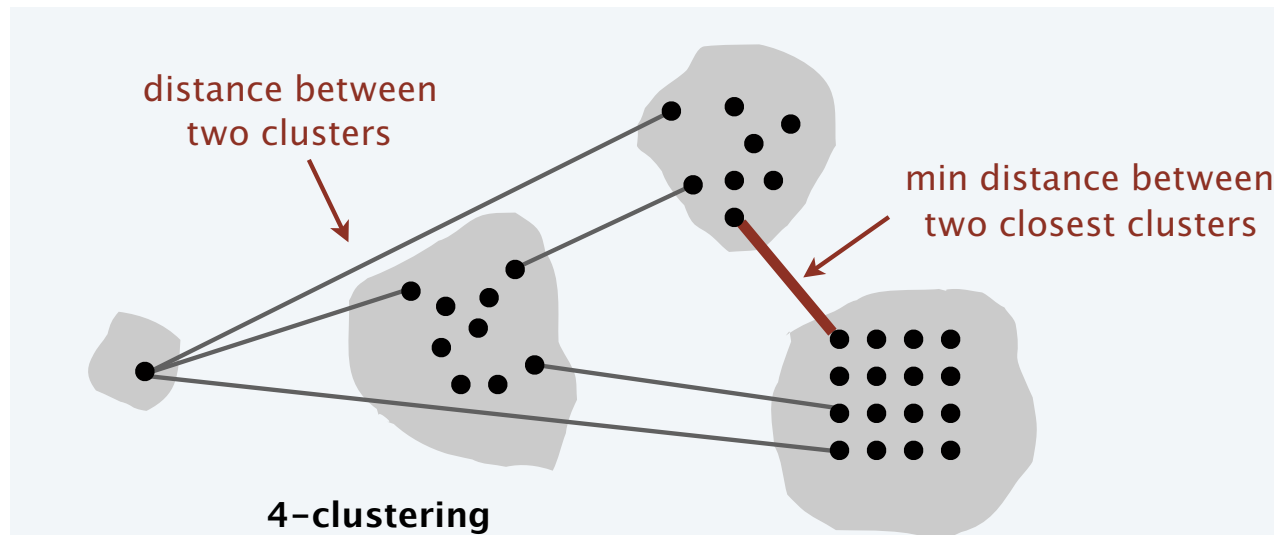


# Borůvka's Algorithm

- Why does this work?
  - Can add edges sequentially and maintain promising set structure
  - Let  $v \in V$  be any vertex in  $G$ . The minimum spanning tree for  $G$  must contain an edge  $(v, w)$  that is the minimum weight edge incident on  $v$ .
- What is hard to compute?
  - A bit of work to do: For each pass, need to avoid edges that form loops
    - Can do in  $O(\log^*(\#V))$  with disjoint set data structure, or other tricks (graph contraction)
- Total complexity  $O(\#E \log(\#V))$

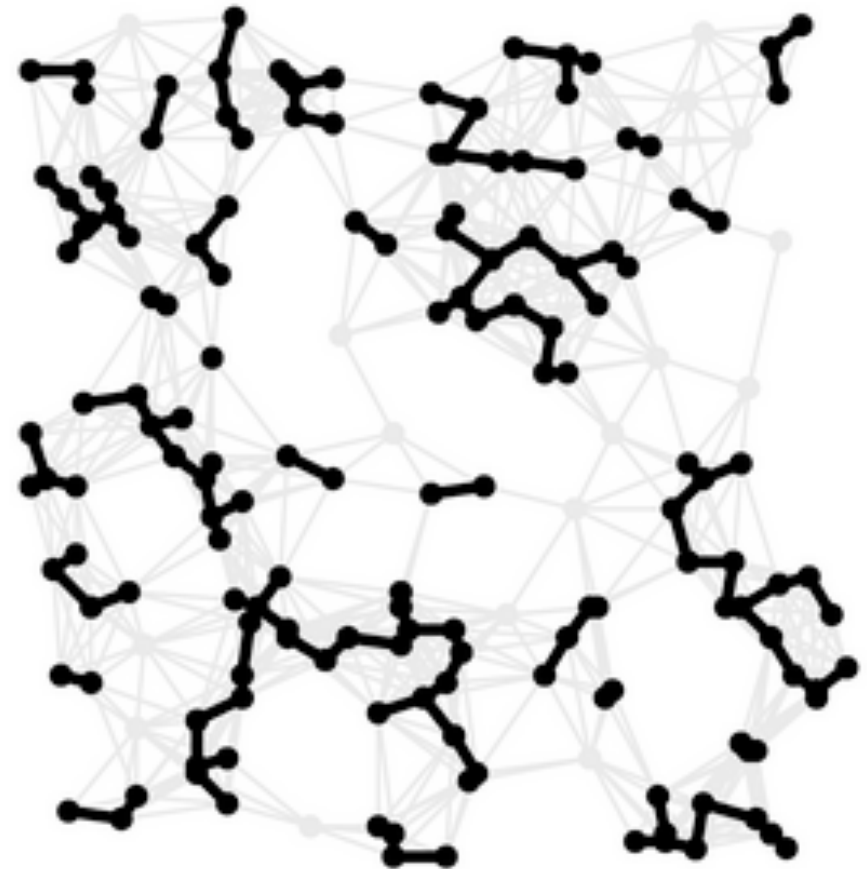
# Application: Clustering

- Given a weighted graph  $G = (V, E)$ , where weight is distance, want to create  $k$  clusters such that we maximize the minimum distance between any pair of vertices in clusters



# Solution

- Approach 1: Run Kruskal's algorithm until there are only  $k$  connected components left
- Approach 2: Run any MST algorithm, and delete the longest  $k-1$  edges in MST



# Why does this work?

- Theorem: Let  $C^* = \{C_1^*, \dots, C_k^*\}$  denote the clustering formed by deleting the  $k - 1$  longest edges of an MST. Then,  $C^*$  is a  $k$ -clustering of max spacing
  - Let  $d^*$  be min distance in  $C^*$ . Consider any other clustering  $C = \{C_1, \dots, C_k\}$
  - Let  $v, w$  be in the same cluster in  $C^*$  (e.g.  $C_1^*$ ) but not in  $C$
  - Then, there is an edge  $\{p, q\}$  on  $v$ - $w$  path in  $C_1^*$ , such that  $p$  in  $C_j$ ,  $q$  in  $C_k$ , and  $j$  differs from  $k$
  - $w(p, q) \leq d^*$  because it was added by Kruskal, so min distance in  $C \leq d^*$

